

Exact Algorithms for Two-Dimensional Knapsack Problems: A Unified Framework with New Benchmark Results

Sunkanghong Wang^{a,d}, Roberto Baldacci^b, Fabio Furini^c, Lijun Wei^{a,*}, Qiang Liu^{a,*} ^a*Guangdong Provincial K*

October 3, 2026

Abstract

This paper investigates two-dimensional knapsack problems modeled as the Generalized Two-Dimensional Knapsack Problem (G2KP), a variant not previously studied in the literature. The importance of the G2KP lies in its crucial role in designing Branch-Price-and-Cut (BPC) algorithms for the classical Two-Dimensional Bin-Packing Problem. We propose a novel and effective exact solution framework that incorporates both non-numerically exact and numerically exact algorithms tailored to G2KP. The framework also achieves state-of-the-art performance on the well-known Two-Dimensional Knapsack Problem (with and without item conflicts) and the Two-Dimensional Orthogonal Packing Problem, solving several previously unsolved instances and establishing new benchmark results. These findings highlight the framework’s potential to inspire algorithmic developments for pricing problems in related BPC methods and higher-dimensional packing problems. To support future research, we make the algorithm code, instance data, and detailed results publicly available.

Keywords: two-dimensional rectangle cutting and packing • knapsack problem • exact algorithm • numerical precision

Funding: This study was supported by the Natural Science Foundation of China [Grants 72271062 and 52575565], the Science and Technology Projects in Guangzhou [Grant 2024A04J01019], and the Province Natural Science Fund of Guangdong [Grants 2025A1515011297 and 2024A1515010246].

1 Introduction

Given a rectangular bin with width $W \in \mathbb{Z}^+$ and height $H \in \mathbb{Z}^+$, and a set $\mathcal{I} = \{1, \dots, n\}$ of n rectangular items, where each item $i \in \mathcal{I}$ has width $w_i \in \mathbb{Z}^+$ and height $h_i \in \mathbb{Z}^+$, we address the following packing problems where items must be placed without overlap or rotation, with their edges aligned parallel to the bin borders—a configuration known as *orthogonal packing*. Figure 1 illustrates an example of such a packing.

- The Two-Dimensional Orthogonal Packing Problem (2OPP). A pure feasibility problem that aims to check if all items in $\mathcal{I}$ can be packed together. In particular, when the sum

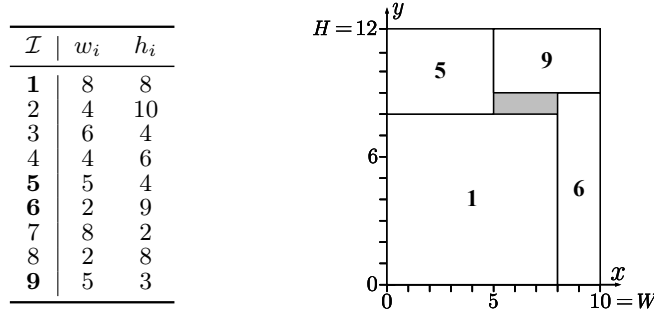


Figure 1: Example of Four Items (shown in bold) Orthogonally Packed into a Bin with $W = 10$ and $H = 12$

of the item areas is exactly equal to the bin area, i.e., $\sum_{i \in \mathcal{I}} w_i h_i = WH$, the problem is called the Perfect Packing Problem (PPP).

- The Two-Dimensional Knapsack Problem (2KP), both with and without item conflicts. A profit $\mu_i \in \mathbb{Z}^+$ is associated with each item $i \in \mathcal{I}$. The problem is determining the maximum profit for packing. In addition, given a set of conflicting pairs of items that cannot be packed together into the bin, the 2KP with conflicts (2KPC) asks for finding a packing of maximum profit, also respecting item conflicts.
- The Generalized 2KP (G2KP). It arises in practice as a subproblem when two-dimensional packing problems, such as the well-known Two-Dimensional Bin Packing Problem (2BPP), are modeled using set-covering formulations extended by *subset-row* (SR) cuts (Jepsen et al., 2008), which involve sets of three items and are particularly effective in column-generation approaches for packing and cutting problems (see, e.g., Wei et al., 2020; Baldacci et al., 2024; Wang et al., 2025d), and are solved via branch-price-and-cut (BPC) with the classical Ryan-Foster (RF) branching rule (Ryan and Foster, 1981), which enforces that two items must be packed either into the same bin or into different ones. Specifically, the G2KP considers two undirected graphs, the *same-bin graph* $\mathcal{G}_s = (\mathcal{I}, \mathcal{E}_s)$ and the *different-bin graph* $\mathcal{G}_d = (\mathcal{I}, \mathcal{E}_d)$. An edge $\{i, j\}$ in $\mathcal{E}_s$ indicates that items i and j must be packed together into the bin. In contrast, an edge $\{i, j\}$ in $\mathcal{E}_d$ signifies that items i and j must not be packed together in the bin, defining a conflict among the items. Additionally, the profit of an item can be zero (i.e., $\mu_i \in \mathbb{Z}^{\geq 0}, \forall i \in \mathcal{I}$). We are also given a subset $\mathcal{T}$ of the index set corresponding to all item triplets. For a triplet index $t \in \mathcal{T}$, let $\mathcal{T}_t \subseteq \mathcal{I}$ denote the corresponding triplet of items, and let $\lambda_t \in \mathbb{Z}^+$ represent an associated penalty. For convenience, we refer to a triplet index simply as a triplet. The objective is to determine a packing that maximizes the sum of profits minus the penalties associated with selected triplets, subject to same-bin and conflict constraints. The penalty λ_t for a triplet $t \in \mathcal{T}$ is incurred only if at least two items from the triplet are packed into the bin. Since G2KP generalizes the above three two-dimensional packing problems, each proved strongly $\mathcal{NP}$ -hard by reduction to the Bin-Packing Problem (Garey and Johnson, 1979), it inherits the same complexity.

Two-dimensional orthogonal cutting and packing problems have been extensively studied in the

Table 1: State-of-the-Art Exact Algorithms for the 2KP(C) and 2OPP

Problem	Short name - Reference	Numerically exact
2KP	CM04 - Caprara and Monaci (2004)	✓
	FSV07 - Fekete et al. (2007)	✓
	BB07 - Baldacci and Boschetti (2007)	
	CLQ20 - de Almeida Cunha et al. (2020)	
2KPC	QHSM17 - de Queiroz et al. (2017)	
2OPP	KINYN09 - Kenmochi et al. (2009)	✓
	MSB12 - Mesyagutov et al. (2012)	
	BR13 - Belov and Rohling (2013)	
	CI18-BB - Côté and Iori (2018)	✓
	CI18-BD - Côté and Iori (2018)	

literature ([Iori et al., 2021](#)). Prominent exact approaches for the 2KP typically follow a two-level method, where items are first selected, and then their feasibility is checked. This category includes a branch-and-bound (BB) algorithm proposed by [Caprara and Monaci \(2004\)](#), featuring four different enumeration schemes. Additionally, [Fekete et al. \(2007\)](#) introduced a tree search algorithm, while [Baldacci and Boschetti \(2007\)](#) developed a cutting plane approach. Recently, based on the two-level method, [de Almeida Cunha et al. \(2020\)](#) proposed a mathematical model integrating advanced reduction techniques. In terms of heuristic methods, several advanced techniques have been developed, among which a recent advancement is the iterative bidirectional heuristic placement algorithm proposed by [Shiangjen et al. \(2018\)](#). The 2KPC was introduced by [de Queiroz et al. \(2017\)](#), who proposed several integer programming formulations solved via branch-and-cut, along with a heuristic combining tabu search and simulated annealing. For the 2OPP, efficient methods include a BB algorithm for the PPP extended with 1×1 units ([Kenmochi et al., 2009](#)), LP-based hybrid techniques ([Mesyagutov et al., 2012](#)), an interval-graph method using bar relaxation ([Belov and Rohling, 2013](#)), and combinatorial BB/primal decomposition ([Côté and Iori, 2018](#)).

To the best of our knowledge, the G2KP has not been studied in the literature. Its importance lies not only in being a generalization of the 2KP(C) and 2OPP, but also in its role in the design of column generation techniques for the 2BPP. In particular, it is central to the development of BPC algorithms, whose effectiveness heavily depends on the iterative solution of the so-called pricing problem during the column generation process. Effective pricing algorithms for the one-dimensional bin-packing problem have already been proposed in the literature (see, e.g., [Wei et al., 2020](#)). However, handling the orthogonal packing constraints in the 2BPP greatly increases the complexity of the pricing problem and the corresponding pricing algorithm. Additionally, a key aspect of BPC algorithms for bin-packing problems is the computation of numerically safe dual bounds with a precision higher than that of general-purpose floating-point solvers ([Baldacci et al., 2024](#)). The recent books by [Desrosiers et al. \(2024\)](#) and [Uchoa et al. \(2024\)](#) provide a comprehensive and up-to-date review of BPC algorithms and related techniques. Table 1 lists leading exact algorithms from the literature for various problems and notes those not based on general-purpose floating-point solvers.

The main contribution of this paper is a novel exact solution framework and efficient non-

numerical and numerical exact algorithms tailored to the G2KP, which is essential for designing BPC algorithms for the 2BPP. We carry out a comprehensive computational study that encompasses benchmark instances and state-of-the-art methods for the 2KP, 2KPC, and 2OPP reported in the literature. The proposed algorithms effectively solve the generated G2KP instances and extend to the 2KP (with or without item conflicts) and the 2OPP, delivering state-of-the-art and new benchmark results. Finally, we provide a publicly available state-of-the-art open-source code for these long-studied packing problems (Wang et al., 2025a).

The remainder of this paper is organized as follows. The next section presents a mathematical formulation for the G2KP. Section 3 describes the proposed exact solution framework and two exact algorithms for solving the G2KP. Section 4 presents and analyzes the computational results of our algorithms. Finally, Section 5 concludes the paper and suggests promising future research directions. Additional material and proofs of statements are provided in the Online Supplement of the paper and in our public repository.

2 A Mathematical Formulation for the G2KP

Hereafter, the horizontal and vertical positions of the bottom-left corner of an item $i \in \mathcal{I}$ are restricted to integer values and represented by (x_i, y_i) , where $x_i \in \mathbb{N} \cap [0, W - w_i]$ and $y_i \in \mathbb{N} \cap [0, H - h_i]$. To reduce the set of potential bottom-left corner positions for item placement within a solution, we consider a subset of all possible positions, referred to as *Meet-In-the-Middle* (MIM) *points*. These MIM points, as introduced by Côté and Iori (2018), help mitigate the problem’s complexity by eliminating redundant positions. Formally, for an item $i \in \mathcal{I}$, we define $\mathcal{X}_i \subseteq \mathbb{Z}_{\geq 0} \cap [0, W - w_i]$ and $\mathcal{Y}_i \subseteq \mathbb{Z}_{\geq 0} \cap [0, H - h_i]$ as the sets of horizontal and vertical MIM points, respectively. Moreover, the set of horizontal MIM points for all items in $\mathcal{I}$ is denoted as $\mathcal{X} = \bigcup_{i \in \mathcal{I}} \mathcal{X}_i$, which aggregates the possible horizontal positions for all items. Further details on the computation of MIM points can be found in Section G.1 of the Online Supplement.

For each item $i \in \mathcal{I}$ and each of its horizontal MIM points $x \in \mathcal{X}_i$, let ϱ_{ix} be a binary variable equal to 1 if and only if the bottom-left corner of the item is placed at position x on the horizontal axis. Additionally, for each item $i \in \mathcal{I}$, let $\hat{y}_i$ be a non-negative integer variable representing its bottom-left corner position on the vertical axis. Given an instance $\boldsymbol{\varrho} \in \{0, 1\}^{\sum_{i \in \mathcal{I}} |\mathcal{X}_i|}$ of the x positions of the items, for each item $i \in \mathcal{I}$, let $\mathcal{X}_i^x$ be the subset of the horizontal MIM points of the item representing the positions where the item also occupies the position x . The set of items packed in the bin and occupying a position $x \in \mathcal{X}$ is as follows:

$$\mathcal{I}(x, \boldsymbol{\varrho}) = \left\{ i \in \mathcal{I} : \sum_{\tilde{x} \in \mathcal{X}_i^x} \varrho_{i\tilde{x}} = 1 \right\} \quad \text{where} \quad \mathcal{X}_i^x = \left\{ x_i \in \mathcal{X}_i : x - w_i + 1 \leq x_i \leq x \right\}. \quad (1)$$

Following the model proposed by Côté et al. (2014) for the Two-Dimensional Strip-Packing

Problem (2SPP), the 2OPP can be formulated as the following *feasibility problem*:

$$(2OPP) \quad \sum_{x \in \mathcal{X}_i} \varrho_{ix} = 1, \quad \forall i \in \mathcal{I}, \quad (2a)$$

$$[\hat{y}_i, \hat{y}_i + h_i) \cap [\hat{y}_j, \hat{y}_j + h_j) = \emptyset, \quad \forall x \in \mathcal{X}, \forall i, j \in \mathcal{I}(x, \varrho), i < j, \quad (2b)$$

$$\varrho_{ix} \in \{0, 1\}, \quad \forall i \in \mathcal{I}, x \in \mathcal{X}_i, \quad (2c)$$

$$\hat{y}_i \in \mathcal{Y}_i, \quad \forall i \in \mathcal{I}. \quad (2d)$$

Constraints (2a) impose that all items are packed into the bin. We note that a necessary and sufficient condition for two items to overlap is that they simultaneously overlap in the x and y positions. Therefore, to prevent overlaps, it is sufficient to ensure that items overlapping in x positions do not overlap in y positions, as stated by conditions (2b). Specifically, (2b) ensure that for each position $x \in \mathcal{X}$, the vertical intervals $[\hat{y}_i, \hat{y}_i + h_i)$, corresponding to items occupying the same position x , do not overlap.

A mathematical formulation for the 2KP can be derived from model (2), replacing equations (2a) with corresponding inequalities and introducing the objective of maximizing the profits of the packed items:

$$(2KP) \quad \max \quad \sum_{i \in \mathcal{I}} \sum_{x \in \mathcal{X}_i} \mu_i \varrho_{ix}, \quad (3a)$$

$$\text{s. t.} \quad \sum_{x \in \mathcal{X}_i} \varrho_{ix} \leq 1, \quad \forall i \in \mathcal{I}, \quad (3b)$$

$$(2b) - (2d). \quad (3c)$$

Assuming item conflicts are represented by graph $\mathcal{G}_d$, the 2KPC model can be derived from (3) by adding the following constraints imposing conflict restrictions defined by edges in $\mathcal{E}_d$:

$$\sum_{x \in \mathcal{X}_i} \varrho_{ix} + \sum_{x \in \mathcal{X}_j} \varrho_{jx} \leq 1, \quad \forall \{i, j\} \in \mathcal{E}_d. \quad (4)$$

For each triplet $t \in \mathcal{T}$, let ψ_t be a binary variable that equals one if at least two items from the triplet are selected in the solution and zero otherwise. The G2KP can be modeled as follows:

$$(G2KP) \quad \max \quad \sum_{i \in \mathcal{I}} \sum_{x \in \mathcal{X}_i} \mu_i \varrho_{ix} - \sum_{t \in \mathcal{T}} \lambda_t \psi_t, \quad (5a)$$

$$\text{s. t.} \quad \sum_{x \in \mathcal{X}_i} \varrho_{ix} - \sum_{x \in \mathcal{X}_j} \varrho_{jx} = 0, \quad \forall \{i, j\} \in \mathcal{E}_s, \quad (5b)$$

$$\sum_{x \in \mathcal{X}_i} \varrho_{ix} + \sum_{x \in \mathcal{X}_j} \varrho_{jx} - \psi_t \leq 1, \quad \forall t \in \mathcal{T}, i, j \in \mathcal{T}_t, i < j, \quad (5c)$$

$$(2b) - (2d), (3b), (4), \quad (5d)$$

$$\psi_t \in \{0, 1\}, \quad \forall t \in \mathcal{T}. \quad (5e)$$

Constraints (4) and (5b) stem from the RF branching rule (Ryan and Foster, 1981) and impose the conflict and same-bin constraints modeled by the edge sets $\mathcal{E}_d$ and $\mathcal{E}_s$, while constraints (5c) account for the penalties induced by the set of triplets $\mathcal{T}$, thereby addressing SR cuts (Jepsen et al., 2008).

3 A Decomposition-Based Exact Solution Approach

The proposed approach leverages a decomposition technique inspired by logic-based Benders decomposition (Hooker, 2000). The original problem is decomposed into a so-called Benders master problem (BMP) and a corresponding slave problem (BSP), which are solved iteratively. The BMP provides fixed integer values used by the BSP to find a feasible primal solution; if unsuccessful, *feasibility cuts* are added to the BMP. This process continues until the BSP finds a feasible—and thus optimal—solution.

We formulate the BMP as a tight relaxation of the G2KP, adapted from the parallel processor scheduling problem with contiguity constraints ($P|cont|C_{\max}$)—a relaxation previously explored in cutting problems (Baldacci and Boschetti, 2007; Côté et al., 2014). Each item $i \in \mathcal{I}$ and the bin are divided into vertical unit slices: item slices are treated as jobs with processing time h_i , and bin slices as processors. The goal is to assign contiguous item slices to consecutive processors while minimizing the makespan. We adapt this to instead maximize total job profits minus penalties for selected triplets, add an inequality constraint of the bin height H , and impose different- and same-bin constraints based on graphs $\mathcal{G}_d$ and $\mathcal{G}_s$. The initial BMP is then derived by omitting variables $\hat{y}_i$ from formulation (5) and incorporating contiguity constraints:

$$\text{(BMP)} \quad \max \quad \sum_{i \in \mathcal{I}} \sum_{x \in \mathcal{X}_i} \mu_i \varrho_{ix} - \sum_{t \in \mathcal{T}} \lambda_t \psi_t, \quad (6a)$$

$$\text{s. t.} \quad \sum_{i \in \mathcal{I}} \sum_{x \in \mathcal{X}_i^{\tilde{x}}} h_i \varrho_{ix} \leq H, \quad \forall \tilde{x} \in \mathcal{X}, \quad (6b)$$

$$(2c), (3b), (4), (5b), (5c), (5e), \quad (6c)$$

where constraints (6b) enforce the contiguity constraints, ensuring that the total height of items occupying any position $\tilde{x} \in \mathcal{X}$ does not exceed the bin height H . To our knowledge, while $P|cont|C_{\max}$ has been used to address the 2SPP (see, e.g., Côté et al., 2014), its adaptation to (G)2KP represents a novel approach.

Given an integer solution $\bar{\varrho}$ to the BMP, the corresponding BSP aims to find a feasible G2KP solution by adjusting the y -coordinates of the items, ensuring that no overlap occurs among items. Specifically, it can be derived from the 2OPP constraints (2b) and (2d), and is formulated as the following feasibility problem:

$$\text{(BSP}(\bar{\varrho})\text{)} \quad [\hat{y}_i, \hat{y}_i + h_i] \cap [\hat{y}_j, \hat{y}_j + h_j] = \emptyset, \quad \forall x \in \mathcal{X}, \forall i, j \in \mathcal{I}(x, \bar{\varrho}), i < j, \quad (7a)$$

$$\hat{y}_i \in \mathcal{Y}_i, \quad \forall i \in \bar{\mathcal{I}}, \quad (7b)$$

where $\bar{\mathcal{I}} = \{i \in \mathcal{I} : \sum_{x \in \mathcal{X}_i} \bar{\varrho}_{ix} = 1\}$ denotes the restricted set of items selected in the solution $\bar{\varrho}$. To efficiently solve the BSP, we employ the combinatorial BB algorithm proposed by Côté et al. (2014), which is briefly described in Section G.2 of the Online Supplement. If the problem $\text{BSP}(\bar{\varrho})$ is feasible, a feasible primal solution (hereafter called *pattern*) associated with $\bar{\varrho}$ is obtained. Otherwise,

$$\sum_{i \in \bar{\mathcal{I}}} \sum_{x \in \mathcal{X}_i : \bar{\varrho}_{ix} = 1} \varrho_{ix} \leq |\bar{\mathcal{I}}| - 1, \quad (8)$$

defines the corresponding *feasibility cut* that can be added to the BMP to cut off the (infeasible) solution $\bar{q}$.

3.1 Overview of Exact Algorithms

Designing numerically safe algorithms independent of floating-point LP solvers is essential to avoid inaccuracies that can affect dual bounds, especially in packing and cutting problems (Held et al., 2012; Pessoa et al., 2020). As shown in Table 1, few existing methods are truly numerically safe. Pessoa et al. (2020) and Baldacci et al. (2024) demonstrate that achieving this in column generation for bin packing requires exact integer (fixed-point) algorithms for the pricing problem. We propose two algorithms—one non-numerically exact and one numerically exact—to solve the G2KP using the outlined approach. They serve as performance baselines for each other. Both algorithms apply the reduction techniques from §3.2, compute initial bounds as described in §3.3, and, when necessary, perform additional reductions (§3.2.1) followed by exact enumeration to solve the BMP (§3.1.1–3.1.2). At selected nodes with an integer BMP solution, a logic-based Benders decomposition method is employed locally to accelerate convergence; hence the term Local Benders Decomposition (LBD) (see §3.4), whose efficiency is demonstrated in Section F.6 of the Online Supplement through computational experiments. Other implementation details are given in §3.5–3.6. All reductions and initial bounds are numerically safe, as they avoid floating-point solvers. For solving $\text{BSP}(\bar{q})$ exactly, we use the BB algorithm of Côté et al. (2014), ensuring integer-based safe computation. Both algorithms can also solve the 2KP(C) and 2OPP. For 2KP(C), we initialize $\mathcal{T} = \emptyset$, remove variables ψ and constraints (5c), ignore (5b), and apply (4) only for 2KPC. For 2OPP, we apply the LBD to the full item set $\mathcal{I}$. A predefined time limit is imposed on each algorithm, upon which execution is terminated.

3.1.1 Non-Numerically Exact Algorithm (N-NEA).

The N-NEA solves the BMP using a branch-and-cut (BC) approach, leveraging the solution framework provided by the CPLEX solver (IBM, 2015). The LP relaxation of the BMP model (6), strengthened by additional valid cuts as detailed in §3.5.1, is solved at the root node of the BC. During the BC enumeration, a CPLEX *lazy callback* procedure is invoked whenever an integer solution $\bar{q}$ to the BMP is found. In this callback procedure, we first check the feasibility of $\bar{q}$ by solving the corresponding slave problem $\text{BSP}(\bar{q})$. If $\text{BSP}(\bar{q})$ is feasible, a pattern associated with $\bar{q}$ is obtained and used to update the incumbent solution, with its value serving as a valid lower bound, denoted by LB . Otherwise, we apply the LBD (see §3.4) to attempt to generate a pattern that contains only the items in $\bar{\mathcal{I}}$ by solving the 2OPP for $\bar{\mathcal{I}}$, i.e., by searching for an alternative solution to $\bar{q}$ such that $\text{BSP}(\bar{q})$ becomes feasible. If LBD finds such a pattern, it is used to update LB . Otherwise, all solutions containing the items in $\bar{\mathcal{I}}$ are proven to be infeasible, meaning $\bar{\mathcal{I}}$ is an infeasible item set. In this case, we add the following *covering cut* to BMP to exclude this set: $\sum_{i \in \bar{\mathcal{I}}} \sum_{x \in \mathcal{X}_i} q_{ix} \leq |\bar{\mathcal{I}}| - 1$. Since this cut excludes a combination of items rather than a combination of item positions, it is typically much stronger than the basic cut (8).

3.1.2 Numerically Exact Algorithm (NEA).

The NEA solves the BMP using a specially designed *floating-point-free* method inspired by the A_3 enumeration scheme introduced in [Caprara and Monaci \(2004\)](#). It begins with the initial lower and upper bounds, denoted as LB_0 and UB_0 , on the optimal G2KP solution value, and then iteratively executes a BB algorithm based on the minimal target lower bound $\underline{LB}$ to guide the enumeration for each iteration. Let UB be the best proven upper bound. Specifically, at a generic iteration, the NEA proceeds as follows: (1) $\underline{LB}$ is set to $\max\{LB + 1, UB - \Delta\}$, where the threshold $\Delta = \max\{\min\{\mu_i : i \in \mathcal{I}\}, \lfloor (UB_0 - LB_0)/6 \rfloor\}$ was found to achieve good performance in preliminary experiments. (2) The BB algorithm is performed to search for a feasible solution with cost $z \in [\underline{LB}, UB]$. During the BB enumeration, once such a solution is found, LB is improved to z , and $\underline{LB}$ is updated to $z + 1$. If the improved LB is equal to UB , the optimal solution is obtained and the NEA ends. (3) If LB was improved in the second step, it is proven to be the optimal solution cost, and UB is set to LB , ending the NEA. Otherwise, it is proven that no feasible solution exists with a cost not less than $\underline{LB}$, so UB is reduced to $\underline{LB} - 1$. If the reduced UB is equal to LB , the incumbent solution is proven to be optimal, and the NEA ends. Otherwise, the next iteration is executed. The BB algorithm is outlined as follows:

- Each node maintains two sets, $\bar{\mathcal{I}}_{\text{in}}$ and $\bar{\mathcal{I}}_{\text{out}}$, representing the items that must be packed and those that cannot be packed, respectively. Branching updates $\bar{\mathcal{I}}_{\text{in}}$ and $\bar{\mathcal{I}}_{\text{out}}$ by generating two child nodes: the first item $i \in \mathcal{I} \setminus (\bar{\mathcal{I}}_{\text{in}} \cup \bar{\mathcal{I}}_{\text{out}})$ that has not yet been considered is assigned either to $\bar{\mathcal{I}}_{\text{in}}$ or to $\bar{\mathcal{I}}_{\text{out}}$, respectively.
- Define the G2KP solution value for an arbitrary item subset $\tilde{\mathcal{I}} \subseteq \mathcal{I}$ as $z_{\text{G2KP}}(\tilde{\mathcal{I}}) = \sum_{i \in \tilde{\mathcal{I}}} \mu_i - \sum_{t \in \mathcal{T} : |\mathcal{T}_t \cap \tilde{\mathcal{I}}| \geq 2} \lambda_t$. **For each newly generated (child) node, it is discarded if $z_{\text{G2KP}}(\bar{\mathcal{I}}_{\text{in}}) > UB$ or $\sum_{i \in \bar{\mathcal{I}}_{\text{in}}} w_i h_i > WH$.** Otherwise, two local upper bounds for the node are computed sequentially. The node is deleted if either bound is less than $\underline{LB}$. Then, for the undiscarded node generated by extending $\bar{\mathcal{I}}_{\text{in}}$, if $z_{\text{G2KP}}(\bar{\mathcal{I}}_{\text{in}}) \geq \underline{LB}$, we apply the LBD to attempt to generate a pattern that contains all items in $\bar{\mathcal{I}}_{\text{in}}$. If such a pattern is found, a solution with cost $z_{\text{G2KP}}(\bar{\mathcal{I}}_{\text{in}})$ is obtained; otherwise, all solutions containing the items in $\bar{\mathcal{I}}_{\text{in}}$ are proven infeasible, implying that $\bar{\mathcal{I}}_{\text{in}}$ is an infeasible item set, and the node is removed.

Compared to the A_3 enumeration scheme in [Caprara and Monaci \(2004\)](#), the NEA incorporates additional constraints and dominance relationships between items to improve the update of the sets $\bar{\mathcal{I}}_{\text{in}}$ and $\bar{\mathcal{I}}_{\text{out}}$. Moreover, we design an effective updating method for updating the minimal target lower bound, which is not discussed in their work. We also compute generally stronger local bounds (see §3.6.1 for details).

3.2 Problem Reductions

We apply three optimality-preserving reduction techniques. We first observe that if for an item $i \in \mathcal{I}$, its profit is zero and it is not *connected* to any other item with a non-zero profit, we can remove it from the set $\mathcal{I}$. Formally, the item set $\mathcal{I}$ is reduced to $\{i \in \mathcal{I} : \sum_{j \in \mathcal{S}_i} \mu_j >$

$0\}$, where $\mathcal{S}_i$ denotes the *maximum connected component* of the graph $\mathcal{G}_s$ containing vertex i . Additionally, bin-item resizing techniques are applied, which are widely used in cutting and packing studies (see, e.g., [Boschetti et al., 2002](#); [Côté et al., 2021](#)) due to their broad applicability. Specifically, we iteratively shrink the bin and enlarge the items until no further dimensional adjustments are possible. In each iteration, the width W of the bin is reduced to $\max \{z = \sum_{i \in \mathcal{I}} w_i \vartheta_i : z \leq W, \vartheta_i \in \{0, 1\}, i \in \mathcal{I}\}$. A similar reduction is applied to the bin's height. To enlarge the items, the width w_i of each item $i \in \mathcal{I}$ is increased to $W - \bar{W}_i$, where $\bar{W}_i = \max \{z = \sum_{j \in \mathcal{I} \setminus \{i\}} w_j \vartheta_j : z \leq W - w_i, \vartheta_j \in \{0, 1\}, j \in \mathcal{I} \setminus \{i\}\}$. A similar procedure is used to adjust the heights of the items. All modified bin and item dimensions can be efficiently computed using standard dynamic programming techniques. Below, additional reduction rules are presented.

3.2.1 Reductions Based on Generalized Dominance Conditions.

Building on earlier reductions ([Beasley, 1985b](#); [Hadjiconstantinou and Christofides, 1995](#)), [Boschetti et al. \(2002\)](#) introduced a domination-based technique for 2KP, later adopted by [Baldacci and Boschetti \(2007\)](#). An item $i \in \mathcal{I}$ dominates item $j \in \mathcal{I}$ if $w_i \leq w_j$, $h_i \leq h_j$, and $\mu_i \geq \mu_j$. This allows replacing j with i in any feasible solution without worsening the objective, thus reducing the solution space by, for example, prioritizing i over j . However, their approach cannot be directly applied to G2KP due to the additional constraints in (4), (5b), and (5c), which render the original dominance conditions invalid. We introduce generalized dominance conditions for G2KP to address this limitation, as presented in Dominance 3.1. Specifically, let $\mathcal{D}_i$ represent the set of items that cannot be packed with item i , which is recursively derived from the different edge set $\mathcal{E}_d$ and the maximum connected components of the graph $\mathcal{G}_s$ as $\mathcal{D}_i = \{j \in \mathcal{I} : \{i, j\} \in \mathcal{E}_d\} \cup \left(\bigcup_{j \in \mathcal{D}_i} \mathcal{S}_j\right)$.

Dominance 3.1. Item $i \in \mathcal{I}$ dominates item $j \in \mathcal{I}$ if the following conditions hold:

$$w_i \leq w_j, \quad h_i \leq h_j, \quad \text{and} \quad \mathcal{S}_i = \{i\}, \quad (9a)$$

$$\mu_i - \sum_{j' \in \mathcal{S}_j} \mu_{j'} - \sum_{i' \in \mathcal{D}_i \setminus \{j\}} \mu_{i'} - \sum_{\substack{t \in \mathcal{T}: \\ i \in \mathcal{T}_t, |\mathcal{T}_t \setminus \mathcal{D}_i \setminus \{i, j\}| \geq 1}} \lambda_t \geq 0. \quad (9b)$$

Based on Dominance 3.1, we develop a reduction procedure as follows. Let $\mathcal{C}_i$ and $\mathcal{F}_i$ represent the sets of items dominated by and dominating item $i \in \mathcal{I}$, respectively. We reduce the solution space by enforcing that item i must be packed before any item $j \in \mathcal{C}_i$, and similarly, i must be packed after any item $k \in \mathcal{F}_i$. From this, a set $\hat{\mathcal{S}}_i$ of items that must be packed in any solution containing item i can be obtained by expanding $\mathcal{S}_i$ as $\hat{\mathcal{S}}_i = \{i\} \cup \left(\bigcup_{j \in \hat{\mathcal{S}}_i} (\mathcal{S}_j \cup \mathcal{F}_j)\right)$. Furthermore, a set $\hat{\mathcal{D}}_i$ of items that cannot be packed in any solution containing item i is enhanced from $\mathcal{D}_i$ as $\hat{\mathcal{D}}_i = \left(\bigcup_{j \in \hat{\mathcal{S}}_i} (\mathcal{D}_j \cup \tilde{\mathcal{D}}_j)\right) \cup \left(\bigcup_{j \in \hat{\mathcal{D}}_i} \hat{\mathcal{U}}_j\right)$, where $\tilde{\mathcal{D}}_j = \{j' \in \mathcal{I} : w_{j'} + w_j > W, h_{j'} + h_j > H\}$ is the set of items that cannot be packed with item j due to two-dimensional packing constraints, and $\hat{\mathcal{U}}_j$ is the set of items that cannot be packed in any solution containing no item j , computed recursively as $\hat{\mathcal{U}}_j = \{j\} \cup \left(\bigcup_{k \in \hat{\mathcal{U}}_j} (\mathcal{S}_k \cup \mathcal{C}_k)\right)$. Finally, we compute two item subsets, $\mathcal{I}_{\text{in}}$ and $\mathcal{I}_{\text{out}}$,

both of which are initialized as empty and iteratively updated using the following Reductions 3.2-3.8 for each item $i \in \mathcal{I} \setminus \mathcal{I}_{\text{in}} \setminus \mathcal{I}_{\text{out}}$, such that the G2KP defined on $\mathcal{I}$ is equivalent to the G2KP defined on the reduced set $\mathcal{I} \setminus \mathcal{I}_{\text{out}}$, where $\mathcal{I}_{\text{in}} \subseteq \mathcal{I} \setminus \mathcal{I}_{\text{out}}$ are required to be part of the solution. Therefore, the set $\mathcal{I}$ is reduced to $\mathcal{I} \setminus \mathcal{I}_{\text{out}}$ after completing the iteration process. We define $A(\tilde{\mathcal{I}}) = \sum_{i \in \tilde{\mathcal{I}}} w_i h_i$ as the total area of the items in $\tilde{\mathcal{I}} \subseteq \mathcal{I}$, and $\hat{\mathcal{I}}_i = \mathcal{I}_{\text{in}} \cup \hat{\mathcal{S}}_i$ as the set of items that must be packed in any solution containing item i .

Reduction 3.2 (Free out). If an item $j \in \hat{\mathcal{I}}_i$ also belongs to $\mathcal{I}_{\text{out}}$, then $\mathcal{I}_{\text{out}}$ is updated to $\mathcal{I}_{\text{out}} \cup \hat{\mathcal{U}}_i$.

Reduction 3.3 (Free conflict). If two conflicting items j and k are found in $\hat{\mathcal{I}}_i$ such that $j \in \hat{\mathcal{D}}_k$ or $k \in \hat{\mathcal{D}}_j$, then $\mathcal{I}_{\text{out}}$ is updated to $\mathcal{I}_{\text{out}} \cup \hat{\mathcal{U}}_i$.

Reduction 3.4 (Free area). If the total area of the items in $\hat{\mathcal{I}}_i$ exceeds the bin area (i.e., $A(\hat{\mathcal{I}}_i) > WH$), then $\mathcal{I}_{\text{out}}$ is updated to $\mathcal{I}_{\text{out}} \cup \hat{\mathcal{U}}_i$.

Reduction 3.5 (Free value). If $\sum_{j \in \hat{\mathcal{I}}_i} \mu_j - \sum_{t \in \mathcal{T}: |\mathcal{T}_t \cap \hat{\mathcal{I}}_i| \geq 2} \lambda_t > UB_0$, then $\mathcal{I}_{\text{out}}$ is updated to $\mathcal{I}_{\text{out}} \cup \hat{\mathcal{U}}_i$.

Reduction 3.6 (Free number). Define $\hat{\mathcal{I}}_i^x = \{j \in \hat{\mathcal{I}}_i : h_j + h_k > H, \forall k \in \hat{\mathcal{I}}_i, j \neq k\}$ and $\hat{\mathcal{I}}_i^y = \{j \in \hat{\mathcal{I}}_i : w_j + w_k > W, \forall k \in \hat{\mathcal{I}}_i, j \neq k\}$ as the sets of items that must be positioned either to the left or right, and down or up, of all other items in $\hat{\mathcal{I}}_i$, respectively. Let $\bar{n}^x$ and $\bar{n}^y$ represent upper bounds on the number of items in $\hat{\mathcal{I}}_i$ that can be packed along the x - and y -directions, respectively, computed as $\bar{n}^x = \max \left\{ |\hat{\mathcal{I}}_i^x| + \sum_{j \in \hat{\mathcal{I}}_i \setminus \hat{\mathcal{I}}_i^x} \vartheta_j : \sum_{j \in \hat{\mathcal{I}}_i \setminus \hat{\mathcal{I}}_i^x} w_j \vartheta_j \leq W - \sum_{j \in \hat{\mathcal{I}}_i^x} w_j, \vartheta_j \in \{0, 1\}, j \in \hat{\mathcal{I}}_i \setminus \hat{\mathcal{I}}_i^x \right\}$, and $\bar{n}^y = \max \left\{ |\hat{\mathcal{I}}_i^y| + \sum_{j \in \hat{\mathcal{I}}_i \setminus \hat{\mathcal{I}}_i^y} \vartheta_j : \sum_{j \in \hat{\mathcal{I}}_i \setminus \hat{\mathcal{I}}_i^y} h_j \vartheta_j \leq H - \sum_{j \in \hat{\mathcal{I}}_i^y} h_j, \vartheta_j \in \{0, 1\}, j \in \hat{\mathcal{I}}_i \setminus \hat{\mathcal{I}}_i^y \right\}$, respectively. If either $\sum_{j \in \hat{\mathcal{I}}_i^x} w_j > W$, $\sum_{j \in \hat{\mathcal{I}}_i^y} h_j > H$, or $\bar{n}^x \times \bar{n}^y < |\hat{\mathcal{I}}_i|$, then $\mathcal{I}_{\text{out}}$ is updated to $\mathcal{I}_{\text{out}} \cup \hat{\mathcal{U}}_i$.

Reduction 3.7 (Area program). Given a knapsack with capacity A and an item set $\tilde{\mathcal{I}}$, let $z_{\text{KP}}(A, \tilde{\mathcal{I}})$ represent the optimal solution value of the following 0-1 knapsack problem (KP): $z_{\text{KP}}(A, \tilde{\mathcal{I}}) = \max \left\{ \sum_{j \in \tilde{\mathcal{I}}} \mu_j \vartheta_j : \sum_{j \in \tilde{\mathcal{I}}} w_j h_j \vartheta_j \leq A, \vartheta_j \in \{0, 1\}, j \in \tilde{\mathcal{I}} \right\}$. If $\sum_{j \in \hat{\mathcal{I}}_i} \mu_j - \sum_{t \in \mathcal{T}: |\mathcal{T}_t \cap \hat{\mathcal{I}}_i| \geq 2} \lambda_t + z_{\text{KP}}(WH - A(\hat{\mathcal{I}}_i), \mathcal{I} \setminus \mathcal{I}_{\text{out}} \setminus \hat{\mathcal{I}}_i \setminus \hat{\mathcal{D}}_i) \leq LB_0$, then $\mathcal{I}_{\text{out}}$ is updated to $\mathcal{I}_{\text{out}} \cup \hat{\mathcal{U}}_i$.

Reduction 3.8 (Updating $\mathcal{I}_{\text{in}}$). If $\sum_{j \in \mathcal{I}_{\text{in}}} \mu_j - \sum_{t \in \mathcal{T}: |\mathcal{T}_t \cap \mathcal{I}_{\text{in}}| \geq 2} \lambda_t + z_{\text{KP}}(WH - A(\mathcal{I}_{\text{in}}), \mathcal{I} \setminus \mathcal{I}_{\text{out}} \setminus \mathcal{I}_{\text{in}} \setminus \hat{\mathcal{U}}_i) \leq LB_0$, then the item set $\mathcal{I}_{\text{in}}$ is updated to $\mathcal{I}_{\text{in}} \cup \hat{\mathcal{I}}_i$. Moreover, $\mathcal{I}_{\text{out}}$ is updated to $\mathcal{I}_{\text{out}} \cup \left(\bigcup_{j \in \hat{\mathcal{I}}_i} \hat{\mathcal{D}}_j \right)$.

The primary computational bottleneck in these reductions is the computation of z_{KP} . The reduction's effectiveness also depends on the order in which items are processed. Preliminary tests indicate that processing items in ascending profit order and utilizing the pre-computed function $z_{\text{D-KP}}$ (described next) instead of z_{KP} when $WHn > 10^6$ provides a suitable trade-off between effectiveness and efficiency. The proofs of the correctness of Dominance 3.1 and Reductions 3.2-3.8 can be found in Section A of the Online Supplement.

3.3 Computing Initial Bounds on the Optimal Solution Value of the G2KP

The one-dimensional relaxation of G2KP has been efficiently solved as the pricing problem in several BPC algorithms for the Bin-Packing Problem (see, e.g., Wei et al., 2020; Baldacci et al., 2024). We adopt the algorithm described by Wei et al. (2020) (see Section 3.1 and Algorithm 1 of their work; details are omitted here for brevity). The method follows the structure of classical dynamic programming for the KP, where $|\mathcal{I}|$ stages are considered and each stage $j \in \mathcal{I}$ corresponds to a subset of feasible patterns involving the first j items, each represented by a label. It is implemented in a label-setting fashion, in which labels are expanded only after their feasibility and dominance have been verified.

To improve label updates, we incorporate the dominance rules based on sets $\hat{\mathcal{S}}_i$ and $\hat{\mathcal{U}}_j$ (see §3.2.1): if a label includes item $i \in \mathcal{I}$, it must also include all items in $\hat{\mathcal{S}}_i$ and exclude those in $\hat{\mathcal{D}}_i$; if item i is excluded, all items in $\hat{\mathcal{U}}_i$ are also excluded.

We first compute a fast initial upper bound, $UB_0^{\text{KP}} = z_{\text{D-KP}}(WH)$, where $z_{\text{D-KP}}(A)$ is the following enhanced 0-1 knapsack function, computed using dynamic programming as $z_{\text{D-KP}}(A) = \min_{\epsilon \in \Gamma} \max \left\{ \sum_{i \in \mathcal{I}} \mu_i \vartheta_i : \sum_{i \in \mathcal{I}} f(\epsilon, \frac{w_i h_i}{A}) \vartheta_i \leq 1, \vartheta_i \in \{0, 1\}, i \in \mathcal{I} \right\}$, where $f(\epsilon, a)$ is a dual feasible function proposed by Fekete and Schepers (2001), computed as $f(\epsilon, a) = a$ if $\epsilon = 0$ or $(\epsilon + 1)a \in \mathbb{Z}$, and $f(\epsilon, a) = \frac{\lfloor (\epsilon + 1)a \rfloor}{\epsilon}$ otherwise. Preliminary experiments suggest that setting $\Gamma = \{0, 1, \dots, \min\{4, \lfloor \frac{10^6}{WHn} \rfloor\}\}$ strikes a good balance between computational efficiency and the quality of the bound.

The label-setting algorithm is then employed to compute an initial lower bound by conducting a heuristic 2OPP feasibility check for each non-dominated label at every stage. This check leverages a skyline-based heuristic adapted from Wei et al. (2011) and Wei et al. (2016) (see Section E of the Online Supplement for details). Labels that fail the check are discarded. Specifically, let $\mathcal{V}_l \subseteq \mathcal{I}$ denote the set of items associated with label l . The items in $\mathcal{V}_l$ are sorted in a non-increasing order of area and then passed to the heuristic. A label passes the check if the heuristic successfully packs all items in $\mathcal{V}_l$ into a single bin. Each label that passes corresponds to a feasible G2KP solution, and the best among these is selected as the initial lower bound LB_0 (i.e., the initial solution). If the optimality gap remains, we apply another similar label-setting algorithm, omitting the 2OPP check, to compute a usually tighter **upper bound**, UB_0^{GKP} , by solving the one-dimensional relaxation of the G2KP. The final initial upper

bound, UB_0 , is then set to the smaller of UB_0^{KP} and UB_0^{GKP} . Retaining too many labels can result in memory overflow in the label-setting framework. To address this, we limit each stage to retaining only the top $|\overline{\mathcal{L}}|$ labels with the highest solution cost, breaking ties by selecting the label with the maximum remaining available capacity. Specifically, when the label-setting algorithm computes UB_0^{GKP} , it terminates once the limit is reached. However, in computing LB_0 , the procedure immediately proceeds to the next stage upon reaching the limit, and then $|\overline{\mathcal{L}}|$ is reduced to $\max\{1, \lfloor \varsigma \times |\overline{\mathcal{L}}| \rfloor\}$. Preliminary experiments indicate that $|\overline{\mathcal{L}}| = 4 \times 10^4$ and $\varsigma = 0.8$ yield effective performance.

3.4 Local Benders Decomposition (LBD)

Recall that the LBD is invoked upon detecting the infeasibility of $\text{BSP}(\overline{\boldsymbol{q}})$ in N-NEA, or when $z_{\text{G2KP}}(\overline{\mathcal{I}}_{\text{in}}) \geq \underline{LB}$ in NEA. Given a solution $\overline{\boldsymbol{q}}$ of the BMP with item set $\overline{\mathcal{I}}$, the problem $\text{BSP}(\overline{\boldsymbol{q}})$ checks feasibility along the y -axis for fixed x -positions. Since multiple BMP solutions may correspond to $\overline{\mathcal{I}}$, an infeasible $\text{BSP}(\overline{\boldsymbol{q}})$ does not rule out other infeasible solutions with different x -positions, or a feasible solution can exist with different x -positions. When no feasible solution exists, additional cuts derived from these infeasible configurations can be added to BMP to accelerate convergence. To this end, the LBD method aims to either find a feasible solution or eliminate all BMP solutions associated with $\overline{\mathcal{I}}$ whose BSPs are infeasible, by solving a simpler local variant of BMP (LBMP), formulated as the following feasibility problem:

$$(\text{LBMP}(\overline{\mathcal{I}})) \quad \sum_{x \in \mathcal{X}_i(\overline{\mathcal{I}})} \varrho'_{ix} = 1, \quad \forall i \in \overline{\mathcal{I}}, \quad (10a)$$

$$\sum_{i \in \overline{\mathcal{I}}} \sum_{x \in \mathcal{X}_i^{\tilde{x}}(\overline{\mathcal{I}})} h_i \varrho'_{ix} \leq H, \quad \forall \tilde{x} \in \mathcal{X}(\overline{\mathcal{I}}), \quad (10b)$$

$$\varrho'_{ix} \in \{0, 1\}, \quad \forall i \in \overline{\mathcal{I}}, x \in \mathcal{X}_i(\overline{\mathcal{I}}), \quad (10c)$$

where $\mathcal{X}(\overline{\mathcal{I}})$, $\mathcal{X}_i(\overline{\mathcal{I}})$, and $\mathcal{X}_i^{\tilde{x}}(\overline{\mathcal{I}})$ are the MIM point sets associated with the restricted item set $\overline{\mathcal{I}}$, and $\varrho'_i \in \{0, 1\}^{|\mathcal{X}_i(\overline{\mathcal{I}})|}$, $i \in \overline{\mathcal{I}}$, and ϱ'_{ix} is equal to 1 if item i is packed at position x , and zero otherwise. By enumerating each solution $\overline{\boldsymbol{q}}'$ of $\text{LBMP}(\overline{\mathcal{I}})$ and solving the corresponding $\text{BSP}(\overline{\boldsymbol{q}}')$, one either identifies a pattern containing all items in $\overline{\mathcal{I}}$ or proves that no feasible G2KP solution exists with all items in $\overline{\mathcal{I}}$. Running LBD is effectively equivalent to solving the 2OPP for $\overline{\mathcal{I}}$ via a decomposition-based approach. This typically outperforms the standard BMP-based method, due to: (i) focusing solely on $\overline{\mathcal{I}}$, reducing the solution space; (ii) omitting constraints (6c) and the ψ variables; and (iii) simplifying the problem from optimization to feasibility. A related idea was explored in Yao et al. (2024) for the 2KP with defects; however, their approach relied on fixing variables in BMP rather than simplifying the model, as done in our LBMP.

To efficiently implement LBD, the N-NEA uses logic-based Benders decomposition (LBBD) (see §3.5.2), while the NEA relies on a BB algorithm (see §3.6.2). Before these procedures, both algorithms shrink the bin, enlarge the items in $\overline{\mathcal{I}}$ (see §3.2), and compute the 2SPP lower bounds L_c , L_1 , L_2 (Martello et al., 2003) and $L_{\text{dff}}^{\text{BM}}$ (Boschetti and Montaletti, 2010), thereby allowing early elimination when any of these bounds exceeds the bin height H . A skyline-based tabu search (Online Supplement D) is then used to quickly attempt to pack all items in $\overline{\mathcal{I}}$. In the

special case where $\sum_{i \in \bar{\mathcal{I}}} w_i h_i = WH$, any valid pattern must be waste-free, and LBD reduces to solving the PPP for $\bar{\mathcal{I}}$. In this case, when $|\bar{\mathcal{I}}| \leq 200$ and $\max\{W, H\} \geq 50$, we find that both algorithms are more efficient when they employ a perfect packing algorithm, enhanced from [Kenmochi et al. \(2009\)](#) (see Online Supplement C), rather than LBBD or BB. All 2SPP bounds, tabu search, and perfect packing routines rely only on fixed-point arithmetic and are numerically safe.

3.5 Further Details on the Implementation of N-NEA

3.5.1 Strengthening the BMP.

To accelerate convergence in the BC method for N-NEA, the following valid inequalities are added to the initial BMP model (6) at the BC root node. Their validity follows from simple arguments and is omitted for brevity.

Valid Cut 3.9. These cuts enforce that all items in the set $\mathcal{I}_{\text{in}}$ must be packed:

$$\sum_{x \in \mathcal{X}_i} \varrho_{ix} = 1, \quad \forall i \in \mathcal{I}_{\text{in}}. \quad (11)$$

Valid Cut 3.10. This cut constrains the objective value of the BMP to lie within the interval $[LB_0, UB_0]$:

$$LB_0 \leq \sum_{i \in \mathcal{I}} \sum_{x \in \mathcal{X}_i} \mu_i \varrho_{ix} - \sum_{t \in \mathcal{T}} \lambda_t \psi_t \leq UB_0. \quad (12)$$

Valid Cut 3.11. These cuts impose a conditional inclusion; if any item $j \in \mathcal{C}_i$ is packed, then item i must also be packed. Conversely, if i is excluded, all $j \in \mathcal{C}_i$ must also be excluded:

$$\sum_{x \in \mathcal{X}_i} \varrho_{ix} \geq \sum_{\tilde{x} \in \mathcal{X}_j} \varrho_{j\tilde{x}}, \quad \forall i \in \mathcal{I}, j \in \mathcal{C}_i. \quad (13)$$

Valid Cut 3.12. These symmetry-breaking cuts impose an ordering on equal-sized items linked by the dominance relation. For each item $i \in \mathcal{I}$ and each item $j \in \mathcal{C}_i$ with $w_i = w_j$ and $h_i = h_j$, item i is assigned a horizontal coordinate no smaller than that of item j whenever item j is selected:

$$\sum_{x \in \mathcal{X}_i} x \varrho_{ix} \geq \sum_{x \in \mathcal{X}_j} x \varrho_{jx}, \quad \forall i \in \mathcal{I}, j \in \mathcal{C}_i, w_i = w_j, \text{ and } h_i = h_j. \quad (14)$$

The validity of these cuts follows jointly with Valid Cut 3. If item j is selected, Valid Cut 3 requires item i to be selected as well, and the inequality fixes only their otherwise symmetric horizontal order. If item j is not selected, its coordinate expression is zero and the inequality is automatically satisfied.

Valid Cut 3.13. These cuts strengthen constraints (4) by preventing overlapping placements of items based on two-dimensional packing conflicts:

$$\sum_{x \in \mathcal{X}_i} \varrho_{ix} + \sum_{x \in \mathcal{X}_j} \varrho_{jx} \leq 1, \quad \forall \{i, j\} \in \mathcal{E}_d \cup \{\{i', j'\} : i' < j', i' \in \mathcal{I}, j' \in \tilde{\mathcal{D}}_{i'}\}. \quad (15)$$

Valid Cut 3.14. These cuts enhance constraints (6b) by restricting the total height of items placed in each bin slice $\tilde{x} \in \mathcal{X}$, ensuring it does not exceed the bin height H . The right-hand side is computed via a 0-1 knapsack function that considers item heights:

$$\sum_{i \in \mathcal{I}} \sum_{x \in \mathcal{X}_i^{\tilde{x}}} h_i \varrho_{ix} \leq z_{\text{KP}}^h(\{j \in \mathcal{I} : \mathcal{X}_j^{\tilde{x}} \neq \emptyset\}), \quad \forall \tilde{x} \in \mathcal{X}, \quad (16)$$

where value $z_{\text{KP}}^h(\tilde{\mathcal{I}})$ is computed as $z_{\text{KP}}^h(\tilde{\mathcal{I}}) = \max \left\{ \sum_{j \in \tilde{\mathcal{I}}} h_j \vartheta_j : \sum_{j \in \tilde{\mathcal{I}}} h_j \vartheta_j \leq H, \vartheta_j \in \{0, 1\}, j \in \tilde{\mathcal{I}} \right\}$.

3.5.2 Logic-Based Benders Decomposition for LBD.

LBMP($\bar{\mathcal{I}}$) is also solved using the CPLEX solver. To speed up convergence, we first strengthen LBMP($\bar{\mathcal{I}}$) by adding the following valid inequalities derived from (14) and (16), restricted to the item set $\bar{\mathcal{I}}$.

$$\sum_{x \in \mathcal{X}_i(\bar{\mathcal{I}})} x \varrho'_{ix} \geq \sum_{x \in \mathcal{X}_j(\bar{\mathcal{I}})} x \varrho'_{jx}, \quad \forall i, j \in \bar{\mathcal{I}}, i < j, w_i = w_j, \text{ and } h_i = h_j, \quad (17a)$$

$$\sum_{i \in \bar{\mathcal{I}}} \sum_{x \in \mathcal{X}_i^{\tilde{x}}(\bar{\mathcal{I}})} h_i \varrho'_{ix} \leq z_{\text{KP}}^h(\{j \in \bar{\mathcal{I}} : \mathcal{X}_j^{\tilde{x}}(\bar{\mathcal{I}}) \neq \emptyset\}), \quad \forall \tilde{x} \in \mathcal{X}(\bar{\mathcal{I}}). \quad (17b)$$

In each iteration, if CPLEX finds no feasible solution $\bar{\varrho}'$ to LBMP($\bar{\mathcal{I}}$), it is proven that no pattern exists for $\bar{\mathcal{I}}$, and the LBD terminates. Otherwise, given the solution $\bar{\varrho}'$, we solve BSP($\bar{\varrho}'$) to check y -feasibility. Two cases arise: (a) If BSP($\bar{\varrho}'$) is feasible, a pattern for $\bar{\mathcal{I}}$ is found and returned. (b) If infeasible, the basic feasibility cut (8) for $\bar{\varrho}'$ is strengthened as described below, and the enhanced cuts are added to LBMP($\bar{\mathcal{I}}$) for the next iteration. Our strengthening cut procedure extends that of Côté et al. (2014), which consists of four steps: three heuristic phases to find smaller infeasible subsets, followed by an LP to lift the left-hand side of the basic cut (8). We modify the third step and show its effectiveness through experiments.

- (1) We seek a vertical line $x \in \{x_i : i \in \bar{\mathcal{I}}\}$ that splits the solution vector $\bar{\varrho}'$ into $\bar{\varrho}'_{\text{left}}$ and $\bar{\varrho}'_{\text{right}}$, corresponding to the disjoint sets $\mathcal{I}_{\text{left}}^x = \{i \in \bar{\mathcal{I}} : x_i + w_i \leq x\}$ and $\mathcal{I}_{\text{right}}^x = \{i \in \bar{\mathcal{I}} : x_i \geq x\}$. We test feasibility by solving BSP($\bar{\varrho}'_{\text{left}}$) and BSP($\bar{\varrho}'_{\text{right}}$). Infeasible subsets are retained and recursively split until no further division is possible. Final steps are applied to the resulting set $\hat{\mathcal{J}}$.
- (2) For each infeasible subset $\hat{\mathcal{I}} \in \hat{\mathcal{J}}$, we examine vertical lines one at a time. From $x = 0$, we scan left to right, removing items with left borders on the current line. Then, from $x = W - 1$, we scan right to left, removing items overlapping the line. After each reduction, we solve a BSP to test feasibility. If feasible, removed items are restored; otherwise, we

obtain a smaller infeasible subset.

- (3) For each infeasible subset $\hat{\mathcal{I}} \in \hat{\mathcal{J}}$, we iteratively remove one item at a time in various orders and solve a BSP on the reduced set. If feasibility is restored, the item is reinstated. Following Côté et al. (2014), who used 12 scans (2 deterministic and 10 random based on area and success score), we also perform 12 scans but with 6 orders: non-decreasing area, width, height, perimeter, x -coordinate, and intersection score, plus 6 random scans. The intersection score counts the number of items that overlap with a given item along the x -axis. We prioritize scans based on item size and local intersection, which proved more effective in preliminary tests. Results are discussed in Section F.3 of the Online Supplement.
- (4) For each infeasible subset $\hat{\mathcal{I}} \in \hat{\mathcal{J}}$, we solve an LP to detect additional infeasible positions. Let $\mathcal{H}_i^{\hat{\mathcal{I}}}$ be the set of items in $\hat{\mathcal{I}}$ that overlap with item i . As long as $\mathcal{H}_i^{\hat{\mathcal{I}}}$ remains unchanged for every $i \in \hat{\mathcal{I}}$, moving i within $\hat{\mathcal{I}}$ preserves infeasibility. For each item $i \in \hat{\mathcal{I}}$, define its valid horizontal range as $[l_i, r_i]$, and let $\mathcal{H}_{x_i}^{\hat{\mathcal{I}}}$ be the set of items overlapping with i when placed at $x_i \in [l_i, r_i]$. If $\mathcal{H}_{x_i}^{\hat{\mathcal{I}}} = \mathcal{H}_i^{\hat{\mathcal{I}}}$, then infeasibility persists. The following LP identifies such positions:

$$\max \quad \sum_{i \in \hat{\mathcal{I}}} (r_i - l_i), \quad (18a)$$

$$\text{s. t.} \quad l_j + w_j \geq r_i + 1, \quad \forall j \in \hat{\mathcal{I}}, i \in \mathcal{H}_j^{\hat{\mathcal{I}}}, \quad (18b)$$

$$0 \leq l_i \leq x_i, \quad \forall i \in \hat{\mathcal{I}}, \quad (18c)$$

$$x_i \leq r_i \leq W - w_i, \quad \forall i \in \hat{\mathcal{I}}. \quad (18d)$$

Constraints (18b) state that item $i \in \mathcal{H}_j^{\hat{\mathcal{I}}}$ overlaps with item $j \in \hat{\mathcal{I}}$, where item i is packed in position $x_i \in [l_i, r_i]$ and item j is packed in position $x_j \in [l_j, r_j]$. Constraints (18c)-(18d) ensure that item $i \in \hat{\mathcal{I}}$ is in the bin. Finally, the optimal solution $(\mathbf{l}^*, \mathbf{r}^*)$ obtained by solving this LP model yields the improved Benders feasibility cut $\sum_{i \in \hat{\mathcal{I}}} \sum_{x \in \mathcal{X}_i(\bar{\mathcal{I}}): [l_i^*] \leq x \leq [r_i^*]} \varrho'_{ix} \leq |\hat{\mathcal{I}}| - 1$.

3.6 Further Details on the Implementation of NEA

3.6.1 Branch-and-Bound for BMP.

Recall that $\bar{\mathcal{I}}_{\text{in}}$ and $\bar{\mathcal{I}}_{\text{out}}$ denote, respectively, the sets of items that must be packed and that cannot be packed, associated with a generic node of the enumeration tree. To exploit additional constraints and dominance relationships in updating the item sets $\bar{\mathcal{I}}_{\text{in}}$ and $\bar{\mathcal{I}}_{\text{out}}$, the root node is initialized with $\bar{\mathcal{I}}_{\text{in}} = \mathcal{I}_{\text{in}}$ and $\bar{\mathcal{I}}_{\text{out}} = \emptyset$, because the global preprocessing has already reduced the working item set to $\mathcal{I} \setminus \mathcal{I}_{\text{out}}$. Moreover, consider left- and right-branching for assigning item i . For the node generated from the left branching, $\bar{\mathcal{I}}_{\text{in}}$ is updated to $\bar{\mathcal{I}}_{\text{in}} \cup \hat{\mathcal{S}}_i$ and $\bar{\mathcal{I}}_{\text{out}}$ is updated to $\bar{\mathcal{I}}_{\text{out}} \cup \hat{\mathcal{D}}_i$; for the node generated from the right branching, $\bar{\mathcal{I}}_{\text{in}}$ remains unchanged while $\bar{\mathcal{I}}_{\text{out}}$ is updated to $\bar{\mathcal{I}}_{\text{out}} \cup \hat{\mathcal{U}}_i$.

Regarding bounding, the first local upper bound is based on a 0-1 KP and is computed as:

$$\overline{UB}_1 = z_{\text{G2KP}}(\overline{\mathcal{I}}_{\text{in}}) + \max \left\{ \sum_{i \in \mathcal{I}} \mu_i \vartheta_i : \sum_{i \in \mathcal{I}} w_i h_i \vartheta_i \leq WH - A(\overline{\mathcal{I}}_{\text{in}}), \vartheta_i \in \{0, 1\}, \forall i \in \mathcal{I} \right\}. \quad (19)$$

The second local upper bound is derived by computing the continuous relaxation of a 0-1 KP, considering the item subset $\mathcal{I}' = \mathcal{I} \setminus (\overline{\mathcal{I}}_{\text{in}} \cup \overline{\mathcal{I}}_{\text{out}})$:

$$\overline{UB}_2 = z_{\text{G2KP}}(\overline{\mathcal{I}}_{\text{in}}) + \max \left\{ \sum_{i \in \mathcal{I}'} \mu_i \vartheta_i : \sum_{i \in \mathcal{I}'} w_i h_i \vartheta_i \leq WH - A(\overline{\mathcal{I}}_{\text{in}}), \vartheta_i \in [0, 1], \forall i \in \mathcal{I}' \right\}. \quad (20)$$

Assuming that $\mathcal{I}$ is sorted in non-increasing order by unit profit (μ/wh), the same ordering holds for $\mathcal{I}'$. Hence, this bound can be efficiently computed in $O(|\mathcal{I}'|)$ linear time using a greedy approach. Note that [Caprara and Monaci \(2004\)](#) considered only the second bound. Moreover, their computation did not account for dominance relationships or additional constraints, resulting in bounds that are generally weaker than ours.

3.6.2 Branch-and-Bound for LBD.

The BB algorithm for performing LBD explores the search tree depth-first. Each node maintains two vectors: the LBMP($\overline{\mathcal{I}}$) solution vector $\boldsymbol{g}'$ and the height vector $\boldsymbol{h}$, where h_x represents the total height of the items occupying the bin slice $x \in \{0, 1, \dots, W-1\}$. Initially, at the root node, both $\boldsymbol{g}'$ and $\boldsymbol{h}$ are set to the zero vector, i.e., $\boldsymbol{g}' = \mathbf{0}$ and $\boldsymbol{h} = \mathbf{0}$.

The algorithm proceeds by packing the items sequentially. For a node at level $i \in \overline{\mathcal{I}}$, item i is packed at all feasible x -positions in the set $\tilde{\mathcal{X}}_i = \{x \in \mathcal{X}_i(\overline{\mathcal{I}}) : x + w_i \leq W, h(x') + h_i \leq H, \forall x' \in \{x, x+1, \dots, x+w_i-1\}\}$, thereby generating child nodes. In a child node created by packing item i at position $x \in \tilde{\mathcal{X}}_i$, the solution vector $\boldsymbol{g}'$ is updated by setting $g'_{ix} = 1$, and the height vector $\boldsymbol{h}$ is updated by setting $h_{x'} = h_{x'} + h_i$ for all $x' \in \{x, x+1, \dots, x+w_i-1\}$. All feasible solutions to LBMP($\overline{\mathcal{I}}$) are found at the leaf nodes corresponding to level $|\overline{\mathcal{I}}| + 1$. For pruning, we employ the following two fathoming rules: (1) If there exist two items i and j of the same size in $\overline{\mathcal{I}}$ with $i < j$, all positions x such that $x < x_i$ can be removed from $\tilde{\mathcal{X}}_j$. To apply this rule efficiently, we assume that the items in $\overline{\mathcal{I}}$ are initially sorted in decreasing order of area, with ties broken by decreasing width and further ties broken by decreasing height. This ensures that two items of the same size are adjacent in the set. Consequently, if $w_i = w_{i-1}$ and $h_i = h_{i-1}$, we remove all positions x such that $x < x_{i-1}$ from $\tilde{\mathcal{X}}_i$. (2) If there exists an item j with $j \geq i$ such that $\tilde{\mathcal{X}}_j = \emptyset$, the current node can be pruned because it is impossible to derive a feasible LBMP solution. Since this rule is effective only at deeper levels, we apply the rule only when $i \geq \underline{L}$ to balance computational and pruning efficiency, where $\underline{L}$ is an integer parameter initialized to three and dynamically adjusted during the enumeration process. Specifically, we track the number F of consecutive failures resulting from pruning. Once F exceeds the threshold $\left\lfloor \frac{\sum_{k \in \overline{\mathcal{I}}} |\mathcal{X}_k(\overline{\mathcal{I}})|}{|\overline{\mathcal{I}}|} \right\rfloor$, we increment $\underline{L}$ by one and reset F to zero. The first fathoming rule is taken directly from the BB algorithm for the *bin-packing problem with contiguity constraints* by [Martello et al. \(2003\)](#). It eliminates symmetry by preventing the enumeration of identical partial solutions at multiple nodes in the search tree. The second fathoming rule we propose identifies infeasible nodes in advance, thereby reducing unnecessary exploration.

4 Computational Study

This section presents the computational experiments conducted to evaluate the proposed exact algorithms (N-NEA and NEA) based on 1,447 classic benchmark instances of 2OPP, 2KP, and 2KPC (§4.1), as well as 1,800 newly generated G2KP instances (§4.2). Furthermore, the proposed algorithms are analyzed in Section F of the Online Supplement. The full algorithmic code, instance data, detailed computational results, and solutions are available online at Wang et al. (2025a) to promote future research and fair comparisons.

The proposed algorithms were implemented in JAVA and executed on a Windows 11 Professional operating system equipped with an Intel[®] Core[™] i9-12900H processor (2.5 GHz) and 32 GB of RAM. The processor achieves a single-thread PassMark score of 3,787 (<https://www.cpubenchmark.net>, 2024). These scores serve as a reliable metric for comparing processor performance, with higher scores indicating superior processing capabilities. To ensure a fair comparison with previous algorithms, we adopted the commercial solver CPLEX 12.6.3 as the integer linear programming solver, the earliest version available. All CPLEX defaults were used, except `IntParam.Threads`, which was set to one.

4.1 Results on the 2OPP, 2KP, and 2KPC

The sources and characteristics of benchmark sets for 2OPP, 2KP, and 2KPC are summarized in Table 2, where the column “#I” denotes the number of instances. Specifically: (i) for 2OPP, we employed six sets containing a total of 1,213 instances. The instances in sets C, N, and T are *waste-free*, meaning that each instance admits feasible packings with no unused space; (ii) for 2KP, we employed thirteen sets comprising a total of 72 instances, including the renowned and challenging instance GCUT13, which has remained unsolved for decades; and (iii) for 2KPC, we employed seven sets containing a total of 162 instances. These sets are adapted from seven 2KP benchmark sets (CGCUT(1-3), GCUT(1-13), M(1-5), MW(1-5), NGCUT(1-12), OKP(1-5), and UW(1-11)) by considering a single copy of each item to generate three classes of conflict graphs corresponding to specific graph densities of 10%, 17%, and 25%. Density is the proportion of conflicting item pairs selected uniformly at random.

To maintain consistency with existing computational studies, the time limits for the proposed algorithms when solving 2OPP, 2KP, and 2KPC instances are set to 307, 3660, and 5084 seconds, respectively. These specific time limits are chosen to account for differences in CPU performance. Further details on how to determine them are provided in the “README.md” file in our GitHub repository (Wang et al., 2025a).

Table 3 summarizes the results obtained for these sets, where the columns “Algo”, “#Opt”, and “Time” indicate, for each set, the short name of the best-performing algorithm, the number of instances solved optimally and the average (scaled) computation times in seconds for all instances (both solved and unsolved), respectively. Specifically, for each set, we compare our results with the best-performing approach in the literature. The column “Time_b’” reports the average (scaled) computation time for instances that can be solved by both our approach and the corresponding best-performing method. Every scaled (average) computation time is calculated by multiplying

Table 2: Sources and Characteristics of the Classic Benchmark Sets of 2OPP, 2KP, and 2KPC

Problem	Set	Source	#I	W	H	n	$\{w_i\}$	$\{h_i\}$
2OPP	C	Hopper and Turton (2001)	21	20-160	20-240	16-197	1-72	1-113
	N	Hopper and Turton (2002)	35	200	200	17-197	1-165	1-137
	T	Hopper and Turton (2002)	35	200	200	17-199	1-178	1-180
	CJCM	Clautiaux et al. (2008)	42	20	20	10-23	1-20	2-20
	MSB-450	Mesyagutov et al. (2012)	450	100	100	10-30	1-94	1-83
	MSB-630	Mesyagutov et al. (2012)	630	1000	1000	20	2-1000	3-998
Overall			1213	20-1000	20-1000	10-199	1-1000	1-998
2KP	NGCUT(1-12)	Beasley (1985b)	12	10-30	10-30	7-22	1-30	1-30
	HCCUT(1-5)	Hadjiconstantinou and Christofides (1995)	5	15-40	15-40	7-15	1-30	2-28
	CGCUT(1-3)	Christofides and Whitlock (1977)	3	15-40	10-70	16-62	2-33	1-43
	GCUT(1-13)	Beasley (1985a)	13	250-3000	250-3000	10-50	63-1890	62-970
	OKP(1-5)	Fekete and Schepers (1997)	5	100	100	30-97	1-100	1-99
	WANG3	Wang (1983)	1	40	70	42	9-33	11-43
	H	Herz (1972)	1	98	127	5	13-65	18-54
	HZ(1-2)	Hifi and Zissimopoulos (1996b,a)	2	67-80	78-99	5-6	5-54	6-54
	M(1-5)	Morabito et al. (1992)	5	156-806	100-750	10	32-449	16-881
	MW(1-5)	Hifi (2001)	5	156-806	100-750	10	32-449	16-445
	UU(1-8)	Fayard et al. (1998)	8	500-2024	500-2050	25-55	101-1280	119-1325
	UW(1-11)	Fayard et al. (1998)	11	500-3650	500-3500	25-60	98-2354	92-2245
	W2	Birgin et al. (2012)	1	2769	3427	20	316-2532	451-2007
Overall			72	10-3650	10-3500	5-97	1-2532	1-2245
2KPC	CGCUT(1-3)'	de Queiroz et al. (2017)	9	15-40	10-70	7-20	2-33	1-43
	GCUT(1-13)'	de Queiroz et al. (2017)	39	250-3000	250-3000	10-50	62-970	63-1890
	M(1-5)'	de Queiroz et al. (2017)	15	100-750	156-806	10	16-881	32-449
	MW(1-5)'	de Queiroz et al. (2017)	15	100-750	156-806	10	16-445	32-449
	NGCUT(1-12)'	de Queiroz et al. (2017)	36	10-30	10-30	5-10	1-30	1-30
	OKP(1-5)'	de Queiroz et al. (2017)	15	100	100	15-33	1-100	1-99
	UW(1-11)'	de Queiroz et al. (2017)	33	500-3500	500-3650	25-60	92-2245	98-2354
Overall			162	10-3500	10-3650	5-60	1-2245	1-2354

the corresponding actual (average) computation time by the ratio of the single-thread PassMark score of the processor used for the algorithm to the lowest score among the compared algorithms. The last three columns highlight the key advancements achieved by our approach, including the number of instances solved for the first time ($\#Sf$), the percentage of instances for which our algorithm either found a better solution or matched the best-known solution with shorter computation time ($\uparrow Be$), and the percentage of instances where we matched both the best-known solution and the shortest computation time reported in the literature ($=Eq$). The results can be summarized as follows.

- For 2OPP, the disjunctive scheme proposed by [Mesyagutov et al. \(2012\)](#) is the best-performing method for the MSB-630 set, the combinatorial BB algorithm by [Côté and Iori \(2018\)](#) performs best for the C, N, and T sets, and the primal decomposition method by [Côté and Iori \(2018\)](#) outperforms others for the remaining sets. Additionally, to gather the best-known solutions, we refer to the results of [Kenmochi et al. \(2009\)](#) and [Wei et al. \(2011\)](#). Our approach yielded mixed results, performing well on the first five sets but less well on the last. Notably, the proposed algorithms solve nine previously unsolved instances.
- For 2KP, the BB algorithm using the A_3 enumeration scheme from [Caprara and Monaci \(2004\)](#) performs best on the CGCUT(1-3) set. For the NGCUT(1-12), HCCUT(1-5), OKP(1-5), and WANG3 sets, the tree search algorithm from [Fekete et al. \(2007\)](#) is the most effective. On the GCUT(1-13) set, the cutting plane approach from [Baldacci and Boschetti \(2007\)](#) achieves the best results, while the mathematical model from [de Almeida Cunha et al. \(2020\)](#) excels on the remaining sets. Our approach demonstrates comparable or superior performance across nearly all sets, reducing computation time by an average of

Table 3: Summary of the Results on 2OPP, 2KP, and 2KPC Instances

Problem	Set	#I	Best algorithm from literature				Ours				Deviation			Advancements		
			Algo	#Opt	Time'	Time _b '	Algo	#Opt	Time'	Time _b '	$\Delta\#Opt$	$\Delta Time'$	$\Delta Time_{b'}$	#Sf	$\uparrow Be$	$=Eq$
2OPP	C	21	CI18-BB	8	983.61	227.01	N-NEA	16	418.07	95.38	8	-57%	-58%	0	57%	24%
	N	35	CI18-BB	13	968.65	153.68	(N-)NEA	22	563.17	0.02	9	-42%	-100%	3	34%	37%
	T	35	CI18-BB	12	1006.57	156.11	(N-)NEA	21	585.82	0.01	9	-42%	-100%	6	46%	51%
	CJCM	42	CI18-BD	42	0.19	—	NEA	42	0.19	—	0	-1%	—	0	29%	2%
	MSB-450	450	CI18-BD	450	8.93	—	N-NEA	450	2.29	—	0	-74%	—	0	24%	62%
Overall		1213	MSB12	613	40.14	6.22	N-NEA	564	168.91	19.18	-49	321%	208%	0	52%	8%
2KP	Overall	1213		1138	98.19	15.42		1115	128.97	19.40	-23	31%	26%	9	40%	30%
	NGCUT(1-12)	12	FSV07	12	<0.02	—	NEA	12	0.20	—	0	—	—	0	33%	0%
	HCCUT(1-5)	5	FSV07	5	<0.01	—	NEA	5	0.01	—	0	—	—	0	60%	0%
	CGCUT(1-3)	3	CM04-A ₃	3	179.32	—	NEA	3	9.46	—	0	-95%	—	0	67%	0%
	GCUT(1-13)	13	BB07	12	2807.55	41.51	NEA	12	2776.04	7.38	0	-1%	-82%	0	46%	0%
	OKP(1-5)	5	FSV07	5	26.32	—	NEA	5	136.35	—	0	418%	—	0	40%	0%
	WANG3	1	FSV07	1	1.71	—	NEA	1	0.51	—	0	-70%	—	0	100%	0%
	H	1	CLQ20	1	0.00	—	(N-)NEA	1	0.00	—	0	—	—	0	0%	100%
	HZ(1-2)	2	CLQ20	2	0.00	—	(N-)NEA	2	0.00	—	0	—	—	0	0%	100%
	M(1-5)	5	CLQ20	5	6.42	—	NEA	5	1.03	—	0	-84%	—	0	100%	0%
	MW(1-5)	5	CLQ20	5	5.27	—	NEA	5	0.31	—	0	-94%	—	0	100%	0%
	UU(1-8)	8	CLQ20	2	2701.48	85.41	NEA	8	2568.85	31.87	6	-5%	-63%	6	100%	0%
	UW(1-11)	11	CLQ20	3	31367.61	4950.59	NEA	11	1169.69	4.59	8	-96%	-100%	8	100%	0%
	W2	1	CLQ20	0	41274.00	—	NEA	1	92.54	—	1	-100%	—	1	100%	0%
Overall		72		56	6182.75	912.98		71	976.64	9.77	15	-84%	-78%	15	67%	4%
2KPC	CGCUT(1-3)'	9	QHSM17	9	0.19	—	N-NEA	9	0.00	—	0	-99%	—	0	33%	67%
	GCUT(1-13)'	39	QHSM17	38	150.88	60.11	NEA	38	92.36	0.05	0	-39%	-100%	0	95%	0%
	M(1-5)'	15	QHSM17	15	0.22	—	NEA	15	0.00	—	0	-99%	—	0	60%	40%
	MW(1-5)'	15	QHSM17	15	0.31	—	NEA	15	0.00	—	0	-99%	—	0	60%	40%
	NGCUT(1-12)'	36	QHSM17	36	0.05	—	NEA	36	0.00	—	0	-100%	—	0	42%	58%
	OKP(1-5)'	15	QHSM17	15	12.83	—	NEA	15	0.09	—	0	-99%	—	0	100%	0%
	UW(1-11)'	33	QHSM17	32	144.99	37.02	NEA	33	0.76	0.78	1	-99%	-98%	1	94%	0%
Overall		162		160	67.12	49.55		161	22.40	0.39	1	-67%	-99%	1	73%	24%

Note: (N-)NEA indicates that both NEA and N-NEA achieve the best performance.

Table 4: Summary of Results for Numerically Exact Algorithms on 2OPP and 2KP Instances

Problem	Set	#I	Best numerically exact algorithm from literature				NEA (Ours)			Deviation		
			Algo	#Opt	Time'	Time _b '	#Opt	Time'	Time _b '	$\Delta\#Opt$	$\Delta Time'$	$\Delta Time_{b'}$
2OPP	C	21	CI18-BB	8	983.61	227.01	16	418.16	94.81	8	-57%	-58%
	N	35	CI18-BB	13	968.65	153.68	22	563.17	0.02	9	-42%	-100%
	T	35	CI18-BB	12	1006.57	156.11	21	585.82	0.01	9	-42%	-100%
	CJCM	42	CI18-BB	20	819.08	126.64	42	0.19	0.02	22	-100%	-100%
	MSB-450	450	CI18-BB	415	125.79	14.23	448	8.16	1.71	33	-94%	-88%
Overall		1213		915	369.91	15.30	1094	145.79	1.28	179	-61%	-92%
2KP	NGCUT(1-12)	12	FSV07	12	<0.02	—	12	0.20	—	0	—	—
	HCCUT(1-5)	5	FSV07	5	<0.01	—	5	0.01	—	0	—	—
	CGCUT(1-3)	3	CM04-A ₃	3	179.32	—	3	9.46	—	0	-95%	—
	GCUT(1-13)	13	CM04-A ₃	12	184.01	49.34	12	145.27	7.38	0	-21%	-85%
	OKP(1-5)	5	FSV07	5	26.32	—	5	136.35	—	0	418%	—
Overall		39		38	78.56	49.34	38	66.71	7.38	0	-15%	-78%

Note: In this table, since some benchmark instances are not solved by existing numerically exact algorithms in the literature, their results are not reported. Moreover, in line with CM04-A₃ on the GCUT(1-13) benchmark set, we consider the computation time of the unsolved GCUT13 instance to be 1800 seconds.

84% compared to these leading methods and solving fifteen previously unsolved instances. The highly challenging instance (GCUT13) remains unsolved; however, our approach finds a better solution with a value of 8,661,108, surpassing the best-known value of 8,622,498 reported by [Fekete et al. \(2007\)](#). We also obtain a solution with value 8,743,005 for GCUT13 by extending the time limit (see §F.1 of the Online Supplement).

- For 2KPC, the best-performing method is the F3 approach proposed by [de Queiroz et al. \(2017\)](#). Our approach solves one previously unsolved instance, resulting in a significant 67% reduction in computation time across all instances. It achieves nearly a 100% reduction in computation time across all benchmark sets except the second one. Furthermore, the overall improvement rate is notable, with a 73% performance gain, and in 24% of the instances, our method matches the best-performing approach.

Furthermore, Table 4 compares the state-of-the-art numerically exact algorithms with our NEA. It is worth noting that the last seven 2KP benchmark sets and all 2KPC benchmark sets are not included in this table because they have not been solved by any numerically exact algorithms in the literature. Overall, our NEA significantly outperforms existing numerically

Table 5: Comparison of N-NEA and NEA under Different Time Limit Settings

		Setting 1 (307, 3660, and 5084 seconds)						Setting 2 (1200 seconds)			
			N-NEA		NEA			N-NEA		NEA	
Problem	Set	#I	#Opt	Time	#Opt	Time	TimeLt	#Opt	Time	#Opt	Time
2OPP	C	21	16	88.759	16	88.778	307 s	16	300.327	16	300.346
	N	35	22	119.564	22	119.564		22	450.408	22	450.408
	T	35	21	124.372	21	124.372		21	480.991	21	480.991
	CJCM	42	42	0.047	42	0.040		42	0.047	42	0.040
	MSB-450	450	450	0.485	448	1.732		450	0.485	448	5.701
	MSB-630	630	564	35.859	545	41.842		568	126.151	546	161.078
Overall		1213	1115	27.381	1094	30.951		1119	97.775	1095	117.850
2KP	NGCUT(1-12)	12	12	0.094	12	0.010	3660 s	12	0.094	12	0.010
	HCCUT(1-5)	5	5	0.005	5	0.000		5	0.005	5	0.000
	CGCUT(1-3)	3	3	1.233	3	0.500		3	1.233	3	0.500
	GCUT(1-13)	13	12	282.833	12	281.890		12	93.258	12	92.668
	OKP(1-5)	5	5	619.337	5	7.201		4	260.922	5	7.201
	WANG3	1	1	0.094	1	0.027		1	0.094	1	0.027
	H	1	1	0.000	1	0.000		1	0.000	1	0.000
	HZ(1-2)	2	2	0.000	2	0.000		2	0.000	2	0.000
	M(1-5)	5	5	0.092	5	0.054		5	0.092	5	0.054
	MW(1-5)	5	5	0.073	5	0.017		5	0.073	5	0.017
	UU(1-8)	8	8	444.531	8	135.667		7	197.025	8	135.667
	UW(1-11)	11	10	347.116	11	61.774		10	123.457	11	61.774
	W2	1	1	12.015	1	4.887		1	12.015	1	4.887
Overall		72	70	196.747	71	76.004		68	75.958	71	41.839
2KPC	CGCUT(1-3)'	9	9	0.004	9	0.005	5084 s	9	0.004	9	0.005
	GCUT(1-13)'	39	38	131.887	38	130.428		38	31.934	38	30.840
	M(1-5)'	15	15	0.026	15	0.002		15	0.026	15	0.002
	MW(1-5)'	15	15	0.023	15	0.003		15	0.023	15	0.003
	NGCUT(1-12)'	36	36	0.003	36	0.000		36	0.003	36	0.000
	OKP(1-5)'	15	15	19.070	15	0.128		15	19.070	15	0.128
	UW(1-11)'	33	33	101.056	33	1.073		32	81.759	33	1.073
Overall		162	161	54.107	161	31.630		160	26.114	161	7.656

exact methods, achieving 61% and 15% reductions in computational time on 2OPP and 2KP instances, respectively. In addition, it solves 179 more 2OPP instances than CIBB-18 (Côté and Iori, 2018). On a set of waste-free benchmark instances, our PPP algorithm achieves an average computation-time reduction of approximately 98% compared with the state-of-the-art PPP method (Kenmochi et al., 2009), demonstrating a significant improvement. Further details, results, and comparisons can be found in our GitHub repository.

Finally, to facilitate future method comparisons, we also run our algorithms on these benchmark instances with a fixed common time limit of 1200 seconds and report the results for the previous setting and for this one in Table 5, including the original average computation time (“Time”). The time limit for each problem is given in the column “TimeLt”. It can be seen that, under the same settings, N-NEA performs slightly better in solving 2OPP, whereas NEA has an advantage in solving 2KP and 2KPC: it not only solves a few additional instances but also significantly reduces the computation time. Under different settings, for 2OPP, extending the time limit from 307 seconds to 1200 seconds results in only a few additional instances being solved. For 2KP and 2KPC, reducing the time limit from 3660 seconds and 5084 seconds to 1200 seconds results in N-NEA solving a few fewer instances, whereas NEA does not. This further demonstrates NEA’s high efficiency in solving 2KP(C) and, more importantly, ensures numerical accuracy.

4.2 Results on the G2KP

Since no G2KP benchmark instances exist, we generate them using two 2BPP benchmark sets: 500 classic instances from Berkey and Wang (1987) and Martello and Vigo (1998) (hereafter referred to as the BMWV set) and 500 adapted larger-scale instances (hereafter referred to as the BMWV⁺ set). The 500 instances in the BMWV set are divided into 10 classes, containing

20, 40, 60, 80, and 100 items, depending on how the bins and items are generated. The BMWV⁺ set, generated using the method described in Section B of the Online Supplement, consists of 10 additional challenging classes characterized by larger bins and smaller items, along with five larger values of n given by $\{100, 200, 300, 400, 500\}$.

To generate G2KP instances from practice, we implemented a standard BPC algorithm for solving 2BPP, based on the set-covering formulation strengthened by SR cuts involving item triplets (Jepsen et al., 2008). This algorithm applies the RF branching rule (Ryan and Foster, 1981) and explores the solution space in a depth-first manner. To ensure numerically safe computations, we follow the approach of Baldacci et al. (2024) by enlarging the original floating-point dual solution by a factor of 10^{11} and rounding it down to obtain an integer dual solution. Thus, the corresponding pricing problem is equivalent to the G2KP and is solved by NEA. For each instance in the BMWV set, we use the BPC algorithm to solve it and then record the most time-consuming pricing problem instance as a G2KP instance. In this way, we collected a set of 500 G2KP instances, referred to as the CLA-1 set. A similar process was applied to the BMWV⁺ set, resulting in another set of 500 G2KP instances, referred to as the NEW-1 set. The characteristics of the two resulting sets are summarized in the left half of Table 6. However, we observed that the density of the graphs $\mathcal{G}_d$ and $\mathcal{G}_s$ of the instances in these two sets is very small (i.e., they contain very few edges). This indicates that pricing problems with fewer additional constraints are typically more challenging to solve.

To generate G2KP instances with a larger density of $\mathcal{G}_d$ and $\mathcal{G}_s$, we modified 2BPP instances by introducing different-bin and same-bin constraints as well as SR cuts. Specifically, given a 2BPP instance, we considered three parameters: $\sigma_d \in \{2\%, 4\%\}$, $\sigma_s \in \{2\%, 4\%\}$, and $\sigma_t \in \{15\%, 30\%\}$. For each possible combination of these parameters, we adapted the 2BPP instance to obtain a G2KP instance by introducing uniformly randomly generated $\lfloor \binom{n}{2} \times n\sigma_d \rfloor$ different-bin constraints, $\lfloor \binom{n}{2} \times n\sigma_s \rfloor$ same-bin constraints, and $\lfloor \binom{n}{3} \times n\sigma_t \rfloor$ SR cuts. The label-setting-based heuristic algorithm (described in §3.3) is used to check the feasibility of the generated G2KP instance. If it is infeasible, the constraints are regenerated. Otherwise, we assign to each item an integer profit chosen uniformly at random in $[0, 161168598346]$, and to each SR cut an integer penalty chosen uniformly in $[330729, 89878705225]$. These ranges are derived from the minimum and maximum profits and penalties observed in the CLA-1 and NEW-1 sets, respectively. In this way, for each first instance in the BMWV set with each n value and each class, we generated 8 G2KP instances, resulting in a set of 400 G2KP instances, referred to as the CLA-2 set. A similar process was applied to the BMWV⁺ set, resulting in another set of 400 G2KP instances, referred to as the NEW-2 set. The characteristics of the two resulting sets are summarized in the right half of Table 6.

Table 7 reports the results of N-NEA and NEA on the G2KP instances in the CLA-1 and CLA-2 sets. The computation time of both algorithms on each instance is limited to 1800 seconds. The “Gap₀(%)” column represents the initial optimality gap, calculated as $100 \times (UB_0 - LB_0)/UB_0$. The “#Opt₀” column indicates the number of instances solved by the initial bounds. The “Time₀” column shows the total computation time (in seconds) for computing the initial bounds and performing problem reductions.

Table 6: Characteristics of the Generated G2KP Benchmark Sets

Set	#I	n	W	H	$\{w_i\}$	$\{h_i\}$	$ \mathcal{E}_d $	$ \mathcal{E}_s $	$ \mathcal{T} $	Set	#I	n	W	H	$\{w_i\}$	$\{h_i\}$	$ \mathcal{E}_d $	$ \mathcal{E}_s $	$ \mathcal{T} $
CLA-1	100	20	10-300	10-300	1-100	1-100	0	0	0-17	CLA-2	80	20	10-300	10-300	1-99	1-99	2-4	2-4	21-42
	100	40	10-300	10-300	1-100	1-100	0-1	0-4	0-39		80	40	10-300	10-300	1-100	1-100	5-10	5-10	55-110
	100	60	10-300	10-300	1-100	1-100	0	0-5	0-468		80	60	10-300	10-300	1-100	1-100	8-17	8-17	93-187
	100	80	10-300	10-300	1-100	1-100	0	0-36	0-308		80	80	10-300	10-300	1-100	1-100	12-25	12-25	135-271
	100	100	10-300	10-300	1-100	1-100	0	0-4	0-351		80	100	10-300	10-300	1-99	1-100	17-34	17-34	179-359
Overall	500	20-100	10-300	10-300	1-100	1-100	0-1	0-36	0-468	Overall	400	20-100	10-300	10-300	1-100	1-100	2-34	2-34	21-359
NEW-1	100	100	20-600	20-600	1-150	1-150	0	0-33	0-208	NEW-2	80	100	20-600	20-600	1-150	1-150	17-34	17-34	179-359
	100	200	20-600	20-600	1-150	1-150	0-1	0-230	0-200		80	200	20-600	20-600	1-150	1-150	39-79	39-79	422-845
	100	300	20-600	20-600	1-150	1-150	0-1	0-220	0-111		80	300	20-600	20-600	1-150	1-150	64-128	64-128	688-1377
	100	400	20-600	20-600	1-150	1-150	0	0-169	0-606		80	400	20-600	20-600	1-150	1-150	90-180	90-180	970-1941
	100	500	20-600	20-600	1-150	1-150	0	0-150	0		80	500	20-600	20-600	1-150	1-150	117-234	117-234	1263-2526
Overall	500	100-500	20-600	20-600	1-150	1-150	0-1	0-230	0-606	Overall	400	100-500	20-600	20-600	1-150	1-150	17-234	17-234	179-2526

For the results on the CLA-1 set, the reduction effect is significant, with an average of approximately 5 items fixed and 13 items excluded, which together account for about 30% of the average number of items (60). The initial bounds demonstrate good quality and computational efficiency, solving approximately 20% of the instances in under 1 second on average. In particular, the quality of the initial lower bound is very high, with the gap between it and the final lower bound computed by NEA being only about 2% on average. Moreover, the two proposed exact algorithms perform comparably, with NEA slightly outperforming N-NEA on average. N-NEA appears to perform better only on instances with $n = 60$.

For the results on the CLA-2 set, since the instances involve many different and same-bin constraints, as well as SR cuts, the reduction effect is slightly diminished. On average, almost no items are fixed, but about 10 can be excluded, representing about 17% of the average number of items. The quality of the initial bound remains very good, solving approximately 37% of the instances in a remarkably short time. NEA solves almost all instances and significantly outperforms N-NEA. This may be because many additional constraints help enhance pruning in the BB algorithm, thereby accelerating the search. In contrast, N-NEA uses a mathematical programming approach to handle these constraints, which is less efficient.

Table 7: Results on the G2KP Instances in CLA-1 and CLA-2 Sets

Set	n	#I	Initial phase						N-NEA				NEA			
			$ Z_{in} $	$ Z_{out} $	LB_0	Gap ₀ (%)	#Opt ₀	Time ₀	LB	Gap(%)	#Opt	Time	LB	Gap(%)	#Opt	Time
CLA-1	20	100	2.3	2.5	11.61	5.07	60	0.00	11.69	0.00	100	0.39	11.79	0.00	100	0.05
	40	100	7.6	6.9	15.36	7.05	23	0.19	15.53	0.61	89	238.06	15.73	0.40	89	200.01
	60	100	5.2	12.7	15.58	12.04	12	0.94	15.74	1.26	84	328.05	15.94	1.72	78	418.71
	80	100	3.8	18.9	14.04	16.02	10	1.65	14.39	5.11	71	570.25	14.57	1.74	73	543.34
	100	100	4.8	21.8	12.51	13.97	6	2.08	12.74	5.16	60	789.16	12.83	1.96	68	626.63
Overall	500		4.7	12.6	13.82	10.83	111	0.97	14.02	2.43	404	385.18	14.17	1.16	408	357.75
CLA-2	20	80	0.0	1.2	4.56	1.89	60	0.00	4.56	0.00	80	0.01	4.56	0.00	80	0.00
	40	80	0.0	3.7	8.22	4.00	44	0.03	8.28	0.11	78	77.02	8.28	0.00	80	0.04
	60	80	0.0	8.5	9.90	7.46	33	0.89	9.98	0.39	77	119.66	9.98	0.00	80	0.93
	80	80	0.0	13.5	12.08	21.18	9	2.05	12.23	6.69	54	674.95	12.25	0.00	80	4.61
	100	80	0.1	20.9	14.33	24.60	2	2.64	15.21	9.43	40	983.37	14.62	0.65	76	110.25
Overall	400		0.0	9.6	9.82	11.83	148	1.12	10.05	3.32	329	371.00	9.94	0.13	396	23.17

Note: The units of the lower bounds (LB_0 and LB) reported in this table are all 10^{11} .

Finally, we tested NEA on huge-scale instances from the NEW-1 and NEW-2 sets, with the results reported in Table 8. The “#LBD” column represents the average number of times the LBD is executed, while the “#LBD_{BB}” column indicates the average number of times the exact BB in the LBD is executed. The “Nodes” column shows the average number of BB nodes generated by NEA. The “Time_{LBD}” and “Time_{LBD_{BB}}” columns represent the average total running times, in seconds, of the LBD and of the BB in the LBD, respectively. The results show that the quality of the computed initial solution and the reduction effect remain good even when n is very large. However, under the half-hour time limit, NEA can solve only about

Table 8: Results on the G2KP Instances in NEW-1 and NEW-2 Sets

Set	n	#I	Initial phase						NEA								
			$ Z_{in} $	$ Z_{out} $	LB_0	Gap ₀ (%)	#Opt ₀	Time ₀	#LBD	#LBD _{BB}	Nodes	LB	Gap(%)	#Opt	Time _{LBD}	Time _{LBD_{BB}}	Time
NEW-1	100	100	4.6	7.8	25.7	11.2	5	2.9	9,097,777.5	1,769.1	0.1	25.8	2.6	53	926.4	719.8	933.6
	200	100	4.6	29.3	32.1	9.5	5	28.2	10,515,492.6	5,546.5	0.1	32.2	6.8	29	1238.1	851.8	1281.1
	300	100	0.6	41.7	12.6	8.3	3	8.3	10,938,156.5	7,863.7	0.2	12.7	5.5	24	1355.6	980.1	1382.2
	400	100	0.6	64.5	14.7	8.2	2	11.8	15,575,226.0	4,101.5	0.1	14.8	5.2	23	1355.2	932.4	1390.6
	500	100	0.8	88.3	2.9	7.2	0	8.4	18,959,438.1	7,551.8	0.2	2.9	4.5	19	1431.0	1030.6	1464.9
Overall	500		2.3	46.3	17.6	8.9	15	11.9	13,017,218.1	5,366.5	0.1	17.7	4.9	148	1261.3	902.9	1290.5
NEW-2	100	80	0.0	7.6	16.3	31.5	2	3.8	354.2	2.6	1.6	16.8	3.0	71	115.1	115.0	262.1
	200	80	0.0	16.4	24.6	45.6	0	9.9	1,966.9	7.0	4.4	25.0	23.0	27	647.0	646.8	1210.7
	300	80	0.0	32.0	31.2	49.2	0	11.1	175.8	7.5	3.9	31.4	27.5	18	774.9	774.8	1419.3
	400	80	0.0	48.4	36.1	53.6	0	11.9	645.6	5.4	3.2	36.3	31.4	14	843.8	843.6	1491.0
	500	80	0.0	61.1	40.5	56.5	0	14.4	592.4	4.7	3.4	41.0	35.3	12	812.7	812.4	1538.6
Overall	400		0.0	33.1	29.7	47.3	2	10.2	747.0	5.4	3.3	30.1	24.1	142	638.7	638.5	1184.3

Note: In this table, the units of lower bounds (LB_0 and LB) reported are 10^{11} , and the units of the number of generated nodes are 10^9 .

17% and 18% of the instances in the two sets, respectively, with average computation times of approximately 1290 seconds and 1184 seconds. Solving instances with $n \geq 200$ is significantly harder than solving instances with $n = 100$. Specifically, for the NEW-1 set, almost all the computation time is spent executing LBD, which is called many times. In contrast, although the BB algorithm in LBD is called significantly fewer times, its execution accounts for about 70% of the total computation time. For the NEW-2 set, LBD accounts for only about 54% of the total computation time, with the remaining time spent almost entirely on the outermost BB algorithm, which generates a large number of nodes. These results indicate that the NEW-1 and NEW-2 sets are sufficiently challenging and can serve as effective benchmarks for future related research.

5 Conclusions

This paper introduced the G2KP, which not only generalizes a class of 2KPs and 2OPP but also plays a crucial role in the design of BPC algorithms for solving the 2BPP. We presented a novel exact solution framework and designed efficient non-numerical and numerical exact algorithms (N-NEA and NEA) for solving G2KP, integrating advanced reduction, relaxation, and enumeration techniques. One of the key contributions of this paper is a thorough computational study encompassing benchmark instances and state-of-the-art methods for the 2KP, 2KPC, and 2OPP found in the literature. Moreover, we introduced a publicly available state-of-the-art open-source code for these long-studied packing problems.

We generated four instance sets for G2KP, comprising a total of 1800 instances, which include bin sizes ranging from 10×10 to 600×600 , item sizes from 1×1 to 150×150 , and item numbers from 20 to 500. In these instances, the NEA significantly outperformed N-NEA on average, effectively solving nearly 86% (928 out of 1080) of the instances with $n \leq 100$ and ensuring numerically safe computation. Even on the large-scale instances with $n \geq 200$, NEA solved 166 out of 720. On the 2OPP, 2KP, and 2KPC benchmark instances, both algorithms demonstrated strong competitiveness against state-of-the-art algorithms and provided new benchmark results. For the 2OPP, 9 previously unsolved instances were solved, with N-NEA slightly outperforming NEA on average. For 2KP and 2KPC, NEA significantly outperformed N-NEA, solving 15 and 1 previously unsolved instances, respectively.

The proposed exact solution framework can be adapted to solve other two-dimensional packing problems, such as the 2SPP and the 2KP with guillotine constraints or defects (Côté et al., 2014; Yao et al., 2024; Wang et al., 2025c). It can also be generalized to address higher-dimensional problems. More importantly, it has the potential to inspire pricing algorithms in BPC algorithms for solving multi-bin packing problems, especially 2BPP (Wang et al., 2025b). These represent interesting future research directions.

Acknowledgments

The authors gratefully acknowledge Jean-François Côté for generously providing their detailed 2OPP results, Thiago Alves de Queiroz for sharing the 2KPC instances, and Iori et al. (2022) for establishing the comprehensive two-dimensional packing library, which provided numerous 2OPP and 2KP benchmark instances that were instrumental to this study. They also sincerely thank the anonymous reviewers and the area and associate editors for their helpful suggestions and thorough reviews. All remaining errors are the authors' responsibility.

References

- Roberto Baldacci and Marco A Boschetti. A cutting-plane approach for the two-dimensional orthogonal non-guillotine cutting problem. *European Journal of Operational Research*, 183(3):1136–1149, 2007.
- Roberto Baldacci, Stefano Coniglio, Jean-François Cordeau, and Fabio Furini. A numerically exact algorithm for the bin-packing problem. *INFORMS Journal on Computing*, 36(1):141–162, 2024.
- John E Beasley. Algorithms for unconstrained two-dimensional guillotine cutting. *Journal of the Operational Research Society*, 36(4):297–306, 1985a.
- John E Beasley. An exact two-dimensional non-guillotine cutting tree search procedure. *Operations Research*, 33(1):49–64, 1985b.
- Gleb Belov and Heide Rohling. LP bounds in an interval-graph algorithm for orthogonal-packing feasibility. *Operations Research*, 61(2):483–497, 2013.
- Judith O Berkey and Pearl Y Wang. Two-dimensional finite bin-packing algorithms. *Journal of the Operational Research Society*, 38(5):423–429, 1987.
- Ernesto G Birgin, Rafael D Lobato, and Reinaldo Morabito. Generating unconstrained two-dimensional non-guillotine cutting patterns by a recursive partitioning algorithm. *Journal of the Operational Research Society*, 63(2):183–200, 2012.
- Marco A Boschetti, Aristide Mingozzi, and Eleni Hadjiconstantinou. New upper bounds for the two-dimensional orthogonal non-guillotine cutting stock problem. *IMA Journal of Management Mathematics*, 13(2):95–119, 2002.
- Marco Antonio Boschetti and Lorenza Montaletti. An exact algorithm for the two-dimensional strip-packing problem. *Operations Research*, 58(6):1774–1791, 2010.
- Alberto Caprara and Michele Monaci. On the two-dimensional knapsack problem. *Operations Research Letters*, 32(1):5–14, 2004.
- Nicos Christofides and Charles Whitlock. An algorithm for two-dimensional cutting problems. *Operations Research*, 25(1):30–44, 1977.
- François Clautiaux, Antoine Jouglet, Jacques Carlier, and Aziz Moukrim. A new constraint programming

- approach for the orthogonal packing problem. *Computers & Operations Research*, 35(3):944–959, 2008.
- Jean-François Côté and Manuel Iori. The meet-in-the-middle principle for cutting and packing problems. *INFORMS Journal on Computing*, 30(4):646–661, 2018.
- Jean-François Côté, Mauro Dell’Amico, and Manuel Iori. Combinatorial Benders’ cuts for the strip packing problem. *Operations Research*, 62(3):643–661, 2014.
- Jean-François Côté, Mohamed Haouari, and Manuel Iori. Combinatorial Benders decomposition for the two-dimensional bin packing problem. *INFORMS Journal on Computing*, 33(3):963–978, 2021.
- Jéssica Gabriela de Almeida Cunha, Vinicius Loti De Lima, and Thiago Alves De Queiroz. Grids for cutting and packing problems: a study in the 2D knapsack problem. *4OR*, 18:293–339, 2020.
- Thiago Alves de Queiroz, Pedro Henrique Del Bianco Hokama, Rafael Crivellari Saliba Schouery, and Flávio Keidi Miyazawa. Two-dimensional disjunctively constrained knapsack problem: Heuristic and exact approaches. *Computers & Industrial Engineering*, 105:313–328, 2017.
- Jacques Desrosiers, Marco Lübbecke, Guy Desaulniers, and Jean Bertrand Gauthier. *Branch-and-Price*. GERAD, HÉC Montréal, 2024.
- Didier Fayard, Mhand Hifi, and Vassilis Zissimopoulos. An efficient approach for large-scale two-dimensional guillotine cutting stock problems. *Journal of the Operational Research Society*, 49(12):1270–1277, 1998.
- Sándor P Fekete and Jörg Schepers. A new exact algorithm for general orthogonal d-dimensional knapsack problems. In Rainer Burkard and Gerhard Woeginger, editors, *Algorithms — ESA 1997*, pages 144–156, Berlin, Heidelberg, 1997. Springer.
- Sándor P Fekete and Jörg Schepers. New classes of fast lower bounds for bin packing problems. *Mathematical Programming*, 91:11–31, 2001.
- Sándor P Fekete, Jörg Schepers, and Jan C Van der Veen. An exact algorithm for higher-dimensional orthogonal packing. *Operations Research*, 55(3):569–587, 2007.
- M.R. Garey and D.S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-completeness*. Mathematical Sciences Series. W. H. Freeman, 1979. ISBN 9780716710448.
- Eleni Hadjiconstantinou and Nicos Christofides. An exact algorithm for general, orthogonal, two-dimensional knapsack problems. *European Journal of Operational Research*, 83(1):39–56, 1995.
- Stephan Held, William Cook, and Edward C Sewell. Maximum-weight stable sets and safe lower bounds for graph coloring. *Mathematical Programming Computation*, 4(4):363–381, 2012.
- JC Herz. Recursive computational procedure for two-dimensional stock cutting. *IBM Journal of Research and Development*, 16(5):462–469, 1972.
- Mhand Hifi. Exact algorithms for large-scale unconstrained two and three staged cutting problems. *Computational Optimization and Applications*, 18:63–88, 2001.
- Mhand Hifi and Vassilis Zissimopoulos. Une amélioration de l’algorithme récursif de herz pour le problème de découpe à deux dimensions. *RAIRO-Operations Research*, 30(2):111–125, 1996a.
- Mhand Hifi and Vassilis Zissimopoulos. A recursive exact algorithm for weighted two-dimensional cutting. *European Journal of Operational Research*, 91(3):553–564, 1996b.
- John N Hooker. *Logic-Based Methods for Optimization: Combining Optimization and Constraint Satisfaction*. Wiley Series in Discrete Mathematics and Optimization. Wiley, 2000. ISBN 9781118031285. URL <https://books.google.com/books?id=1fT6qiih5ygC>.
- Eva Hopper and BCH Turton. An empirical investigation of meta-heuristic and heuristic algorithms for a 2D packing problem. *European Journal of Operational Research*, 128(1):34–57, 2001.

- Eva Hopper and BCH Turton. Problem generators for rectangular packing problems. *Stud. Inform. Univ.*, 2(1):123–136, 2002.
- IBM. *IBM ILOG CPLEX Optimization Studio*. IBM Corporation, Armonk, NY, USA, 2015. URL <https://www.ibm.com/products/ilog-cplex-optimization-studio>.
- Manuel Iori, Vinícius L De Lima, Silvano Martello, Flávio K Miyazawa, and Michele Monaci. Exact solution techniques for two-dimensional cutting and packing. *European Journal of Operational Research*, 289(2):399–415, 2021.
- Manuel Iori, Vinícius Loti de Lima, Silvano Martello, and Michele Monaci. 2DPackLib: a two-dimensional cutting and packing library. *Optimization Letters*, pages 1–10, 2022.
- Mads Jepsen, Bjørn Petersen, Simon Spoorendonk, and David Pisinger. Subset-Row inequalities applied to the vehicle-routing problem with time windows. *Operations Research*, 56(2):497–511, 2008.
- Mitsutoshi Kenmochi, Takashi Imamichi, Koji Nonobe, Mutsunori Yagiura, and Hiroshi Nagamochi. Exact algorithms for the two-dimensional strip packing problem with and without rotations. *European Journal of Operational Research*, 198(1):73–83, 2009.
- Silvano Martello and Daniele Vigo. Exact solution of the two-dimensional finite bin packing problem. *Management Science*, 44(3):388–399, 1998.
- Silvano Martello, Michele Monaci, and Daniele Vigo. An exact approach to the strip-packing problem. *INFORMS journal on Computing*, 15(3):310–319, 2003.
- M Mesyagutov, Guntram Scheithauer, and Gleb Belov. LP bounds in various constraint programming approaches for orthogonal packing. *Computers & Operations Research*, 39(10):2425–2438, 2012.
- RN Morabito, Marcos Nereu Arenales, and VF Arcaro. An and-or-graph approach for two-dimensional cutting problems. *European Journal of Operational Research*, 58(2):263–271, 1992.
- Artur Pessoa, Ruslan Sadykov, Eduardo Uchoa, and François Vanderbeck. A generic exact solver for vehicle routing and related problems. *Mathematical Programming*, 183:483–523, 2020.
- David M Ryan and Brian A Foster. An integer programming approach to scheduling. *Computer Scheduling of Public Transport Urban Passenger Vehicle and Crew Scheduling*, pages 269–280, 1981.
- Kanokwatt Shiangjen, Jeerayut Chaijaruwanich, Wijak Srisujalertwaja, Prakarn Unachak, and Samerkae Somhom. An iterative bidirectional heuristic placement algorithm for solving the two-dimensional knapsack packing problem. *Engineering Optimization*, 50(2):347–365, 2018.
- Eduardo Uchoa, Artur Pessoa, and Lorenza Moreno. Optimizing with Column Generation: Advanced Branch-Cut-and-Price Algorithms (Part I). Technical Report L-2024-3, Cadernos do LOGIS-UFF, Universidade Federal Fluminense, Engenharia de Produção, August 2024. URL <https://optimizingwithcolumngeneration.github.io>.
- PY Wang. Two algorithms for constrained two-dimensional cutting stock problems. *Operations Research*, 31(3):573–586, 1983.
- Sunkanghong Wang, Roberto Baldacci, Fabio Furini, Lijun Wei, and Qiang Liu. Exact algorithms for two-dimensional knapsack problems: A unified framework with new benchmark results, 2025a. URL <https://github.com/INFORMSJoC/2025.1423>. Available for download at <https://github.com/INFORMSJoC/2025.1423>.
- Sunkanghong Wang, Roberto Baldacci, Fabio Furini, Lijun Wei, and Qiang Liu. A highly efficient numerically exact algorithm for two-dimensional bin-packing problems. *Available at SSRN 5515822*, 2025b.
- Sunkanghong Wang, Roberto Baldacci, Qiang Liu, and Lijun Wei. EATKG: An open-source efficient exact algorithm for the two-dimensional knapsack problem with guillotine constraints. *European Journal of Operational Research*, 327:735–753, 2025c.

- Sunkanghong Wang, Shaowen Yao, Hao Zhang, Qiang Liu, and Lijun Wei. Arc-flow formulation and branch-and-price-and-cut algorithm for the bin-packing problem with fragile objects. *Computers & Operations Research*, 173:106878, 2025d.
- Lijun Wei, Wee-Chong Oon, Wenbin Zhu, and Andrew Lim. A skyline heuristic for the 2D rectangular packing and strip packing problems. *European Journal of Operational Research*, 215(2):337–346, 2011.
- Lijun Wei, Hu Qin, Brenda Cheang, and Xianhao Xu. An efficient intelligent search algorithm for the two-dimensional rectangular strip packing problem. *International Transactions in Operational Research*, 23(1-2):65–92, 2016.
- Lijun Wei, Zhixing Luo, Roberto Baldacci, and Andrew Lim. A new branch-and-price-and-cut algorithm for one-dimensional bin-packing problems. *INFORMS Journal on Computing*, 32(2):428–443, 2020.
- Shaowen Yao, Hao Zhang, Qiang Liu, Jiewu Leng, and Lijun Wei. Combinatorial Benders’ decomposition for the constrained two-dimensional non-guillotine cutting problem with defects. *International Journal of Production Research*, 62(23):8299–8325, 2024.